
# HTTP, Wireshark, and Digital Forensics

## Part 1: HTTP traffic contains text information

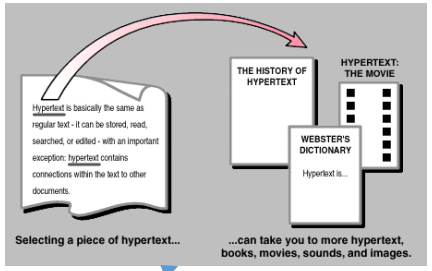
Traffic files: https://drive.google.com/drive/folders/1tiPoLzElH1owRVyIBBoIgNs3Yvfqk2Hh?usp=sharing

# Overview

- Observe HTTP traffic with *curl*
- Review Open Systems Interconnection Model (OSI)
  - Is understanding HTTP enough for forensic investigations?
- Exam HTTP traffics with Wireshark
  - Three-way handshake (TCP)
  - Communication (HTTP request/response)
  - Close connection (TCP)


# Observe HTTP traffic

# What is HTTP?




agreement

- HTTP stands for hypertext transfer protocol and is used to transfer data across the Web

HTTP Request

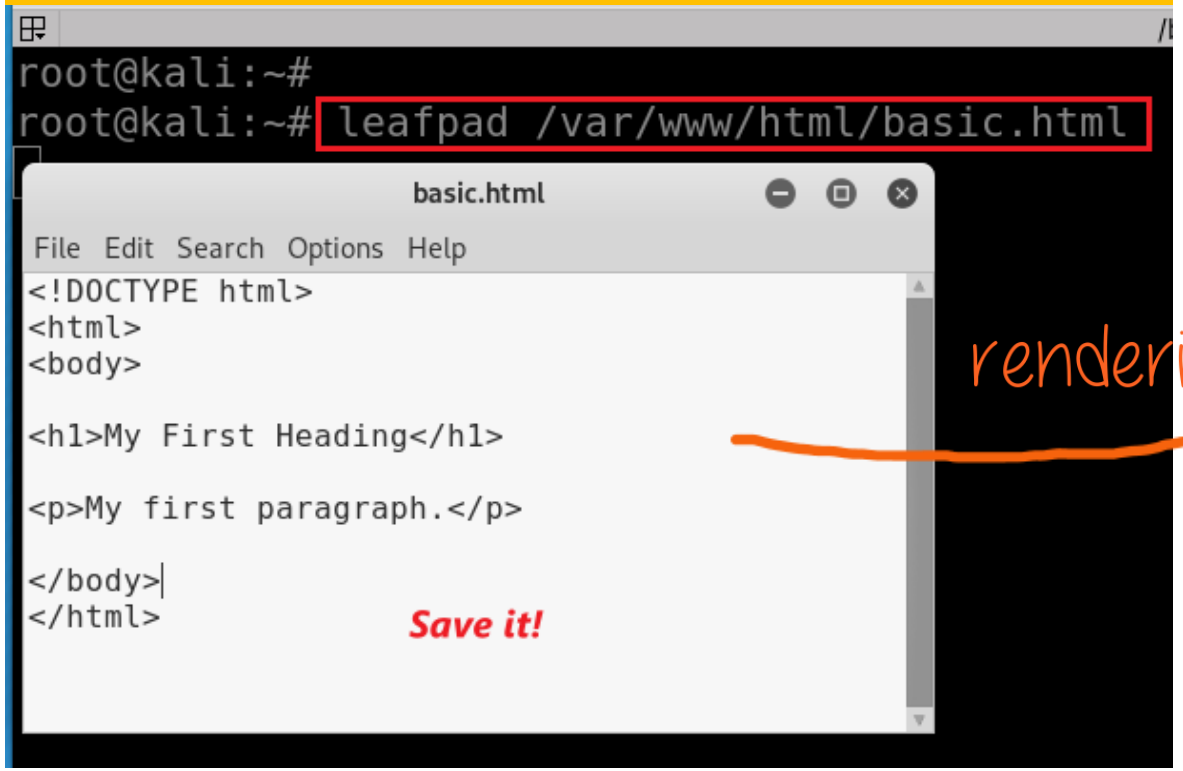
HTTP Response

# Create a simple html page to be observed in Wireshark

(optional) if you are logged in as a non-admin account, such as student, you must change the html folder permission

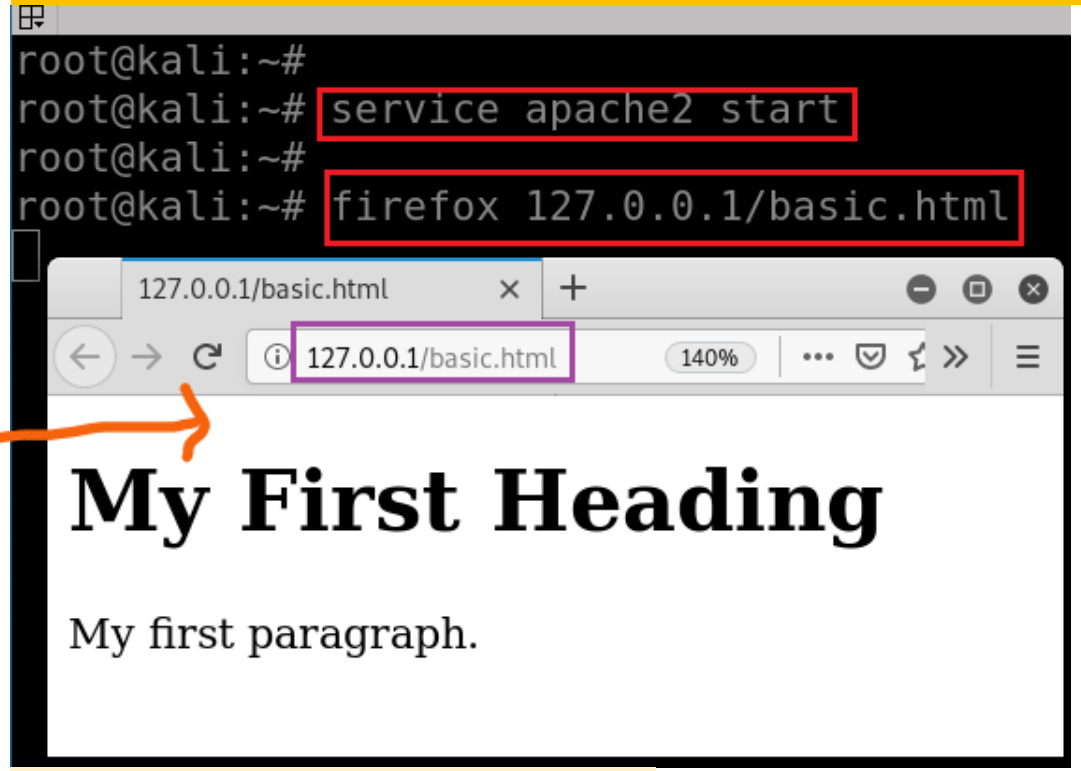
```
┌──(student㉿kali80)-[~/test]
└─$ sudo su
┌──(root💀kali80)-[/home/student/test]
└─# ls -l /var/www/html
total 16
-rw-r--r-- 1 root root 10701 Aug 26 09:17 index.html
-rw-r--r-- 1 root root   612 Aug 26 09:15 index.nginx-debian.html

┌──(root💀kali80)-[/home/student/test]
└─# chmod 777 /var/www/html

┌──(root💀kali80)-[/home/student/test]
└─# exit

┌──(student㉿kali80)-[~/test]
└─$ ls -l   /var/www
total 4
drwxrwxrwx 2 root root 4096 Aug 31 10:51 html
```

- Create a simple webpage, named basic.html, and
- Put it in the folder /var/www/html




rendering

- Start web service
- Show the webpage

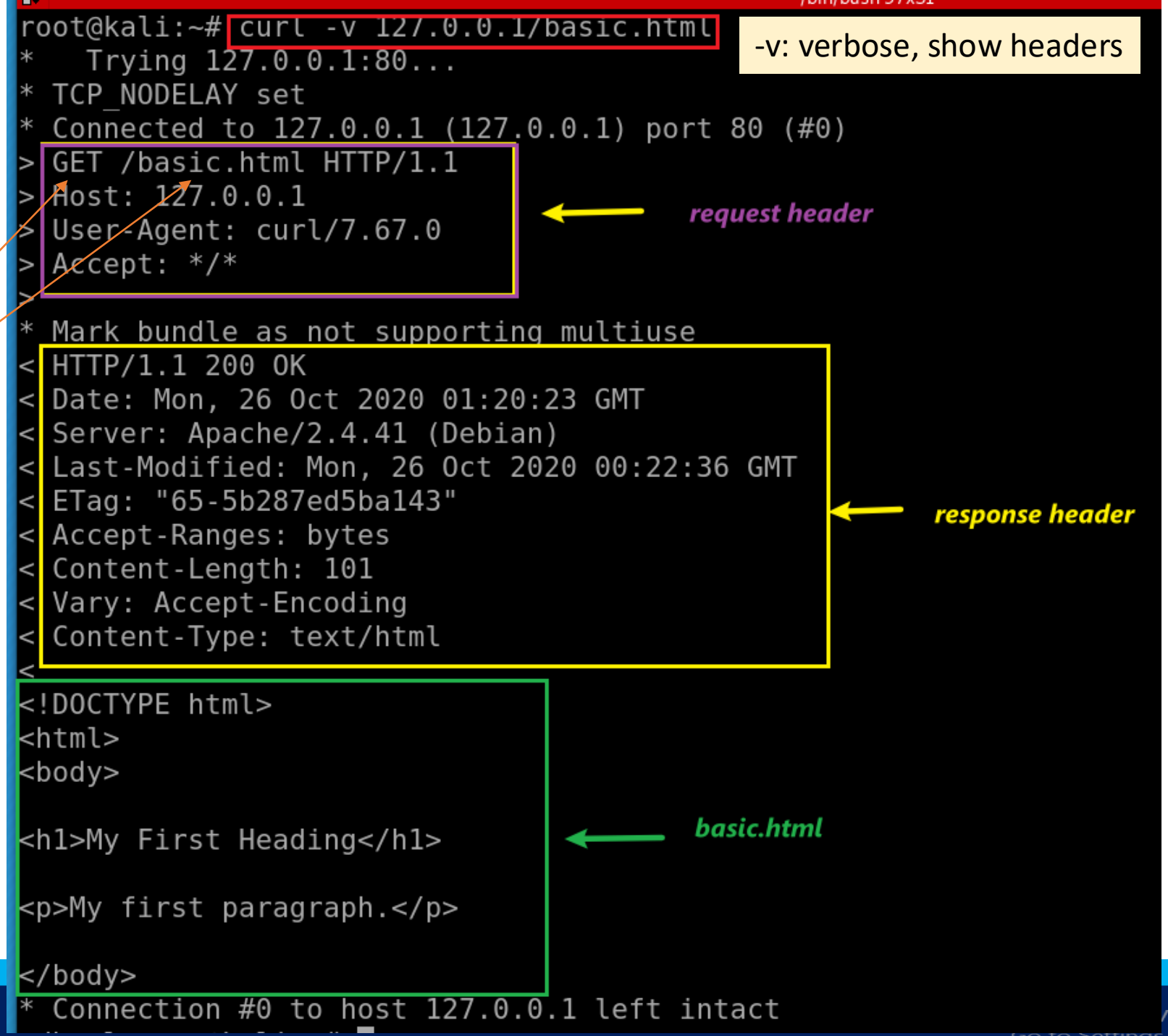


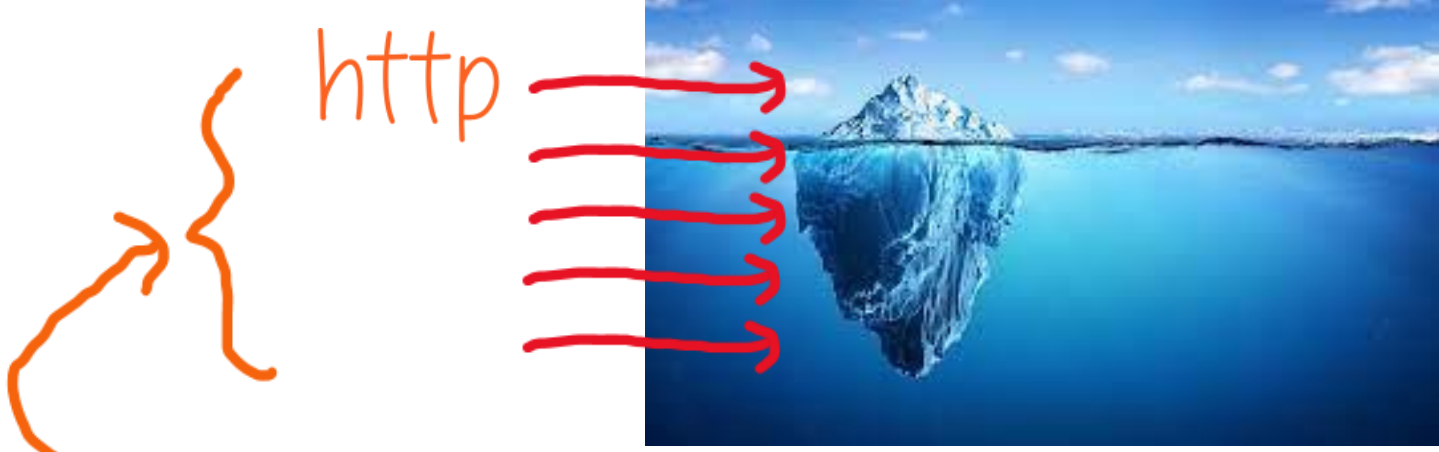

service apache2 stop
service apache2 restart

cURL (pronounced 'curl')

- A command-line tool
- Open source
- Transferring data
- Support many network protocols

Method: Get, POST
Path: /basic.html



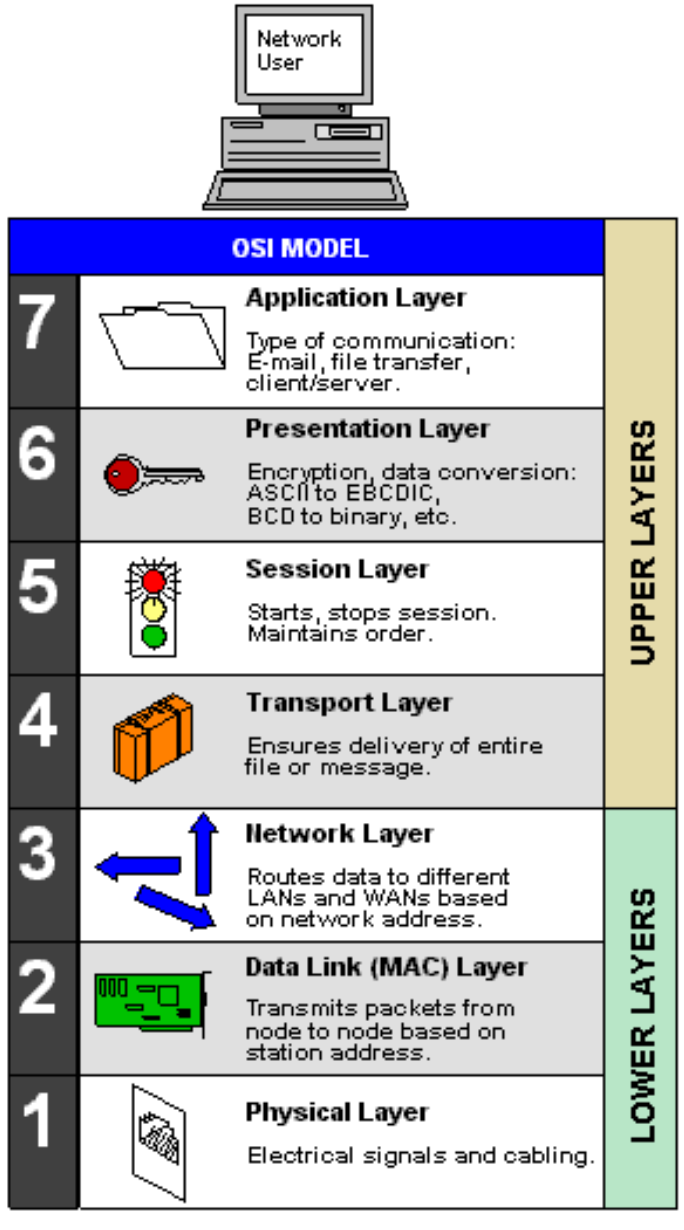



# Review Open Systems Interconnection Model (OSI)

Is understanding HTTP enough for forensic investigations?

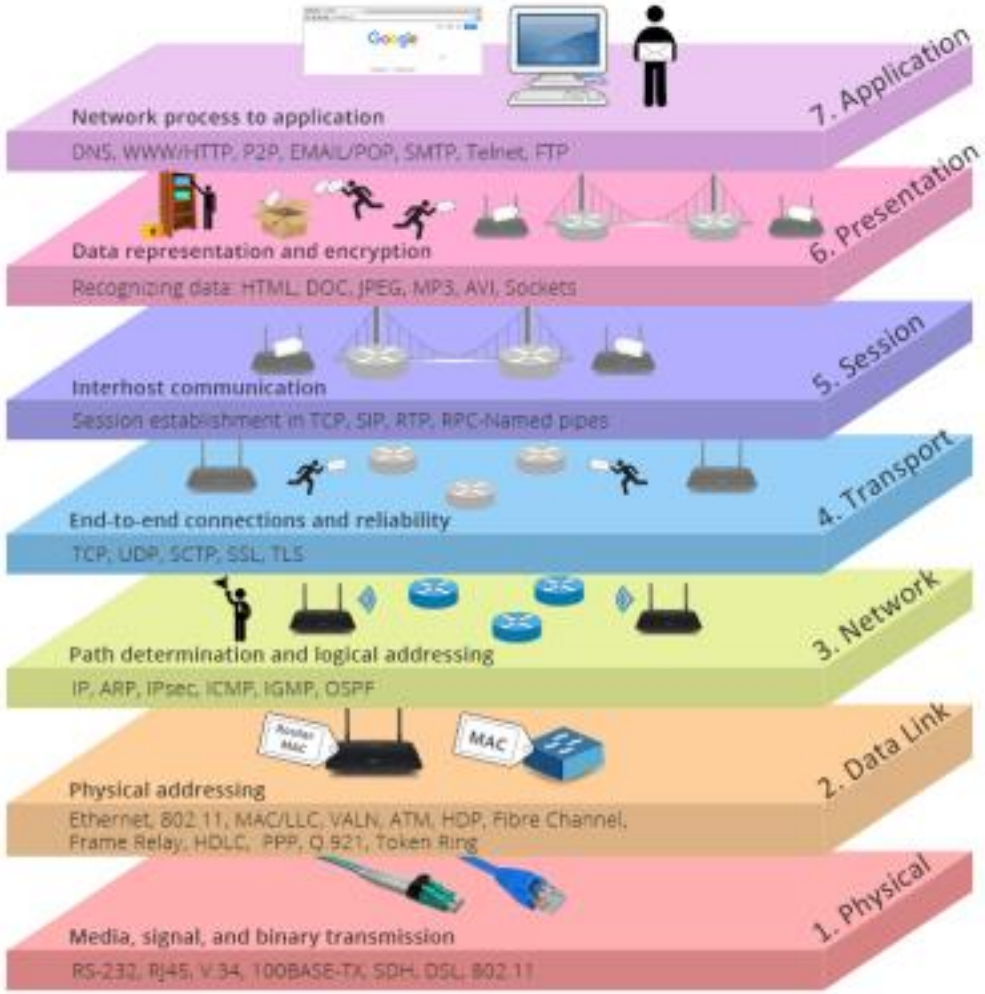
# Importance of understanding underneath of HTTP

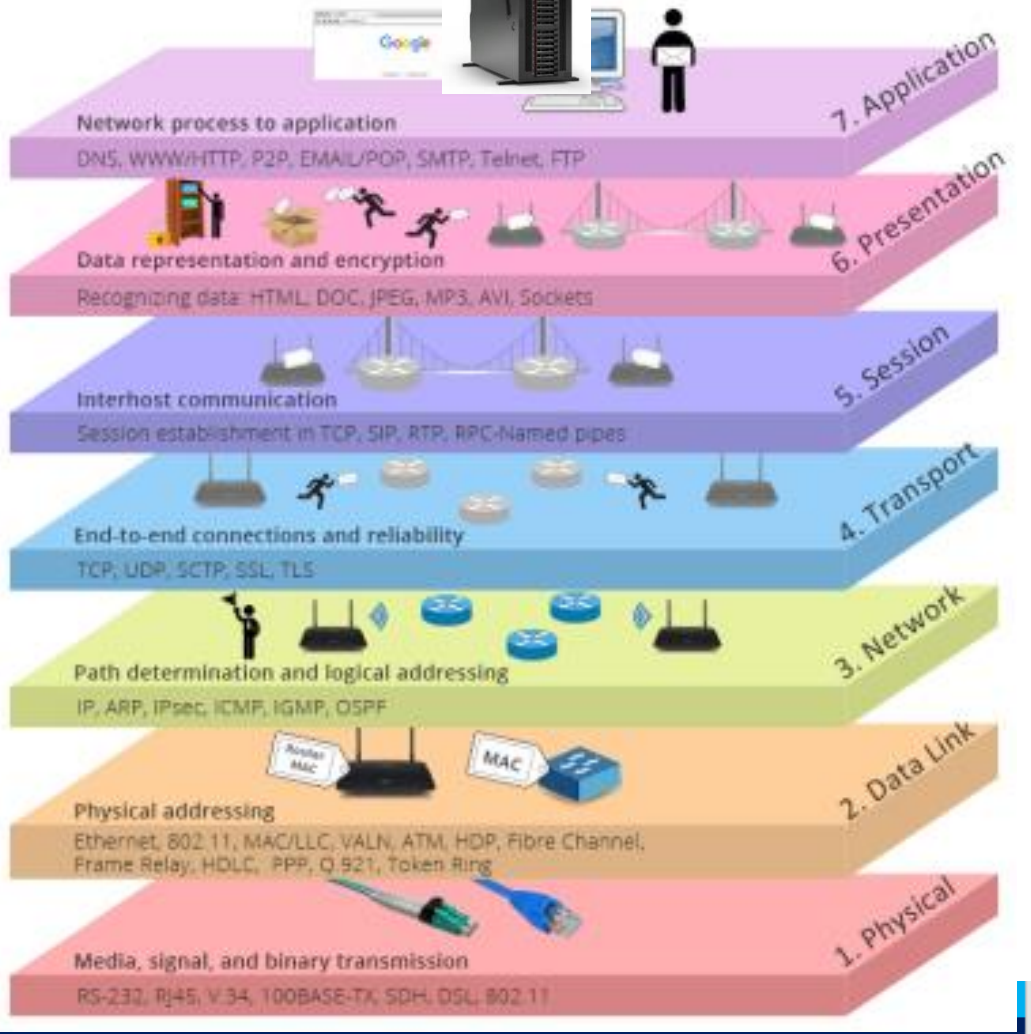
- Many layers (OSI Model) are defined to form computer networks
  - HTTP is one protocol at an application layer
- Forensic investigation focus on all layer
  - Malware analysis
  - Attack analysis
  - File extraction
- The OSI Model (Open Systems Interconnection Model)
  - is a conceptual framework used to describe the functions of a networking system.

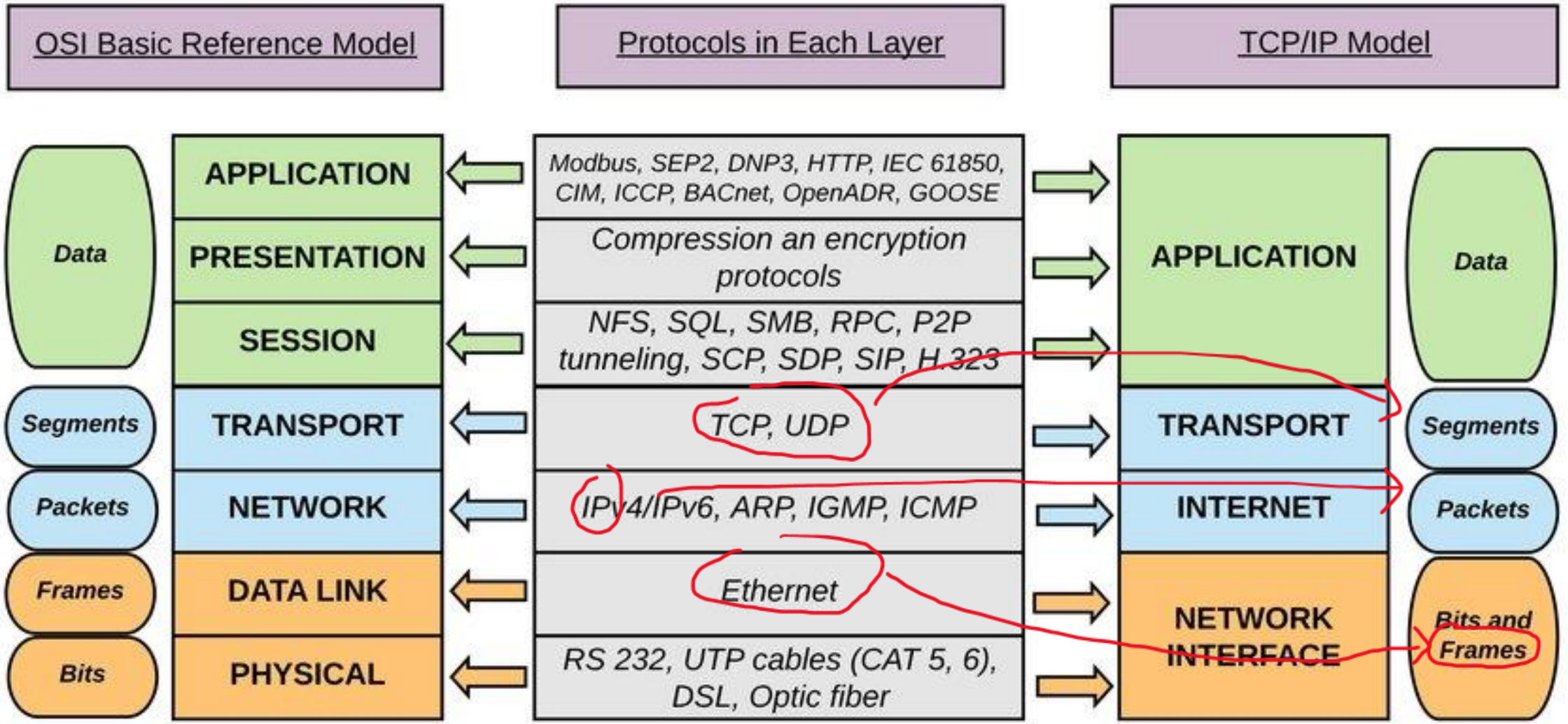


HTTP Request

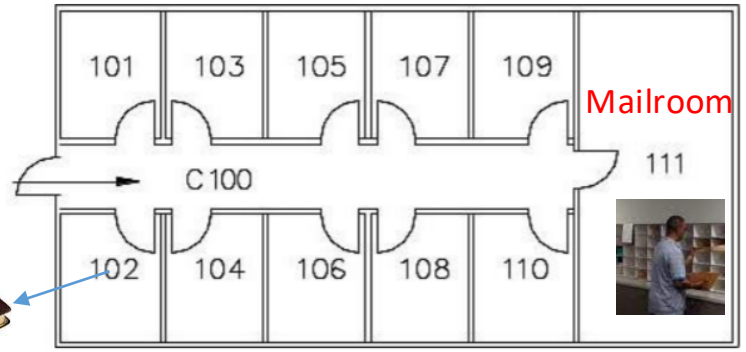
HTTP Response



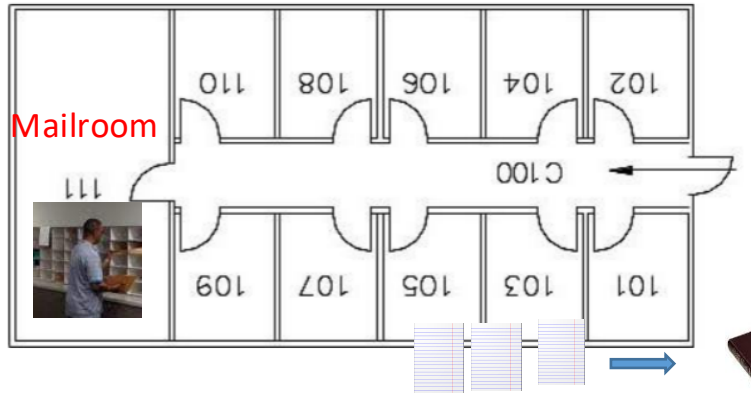


Building A




A->B

Building B





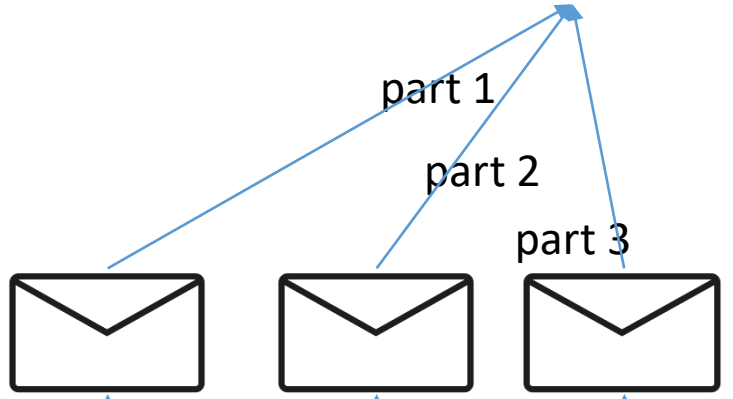


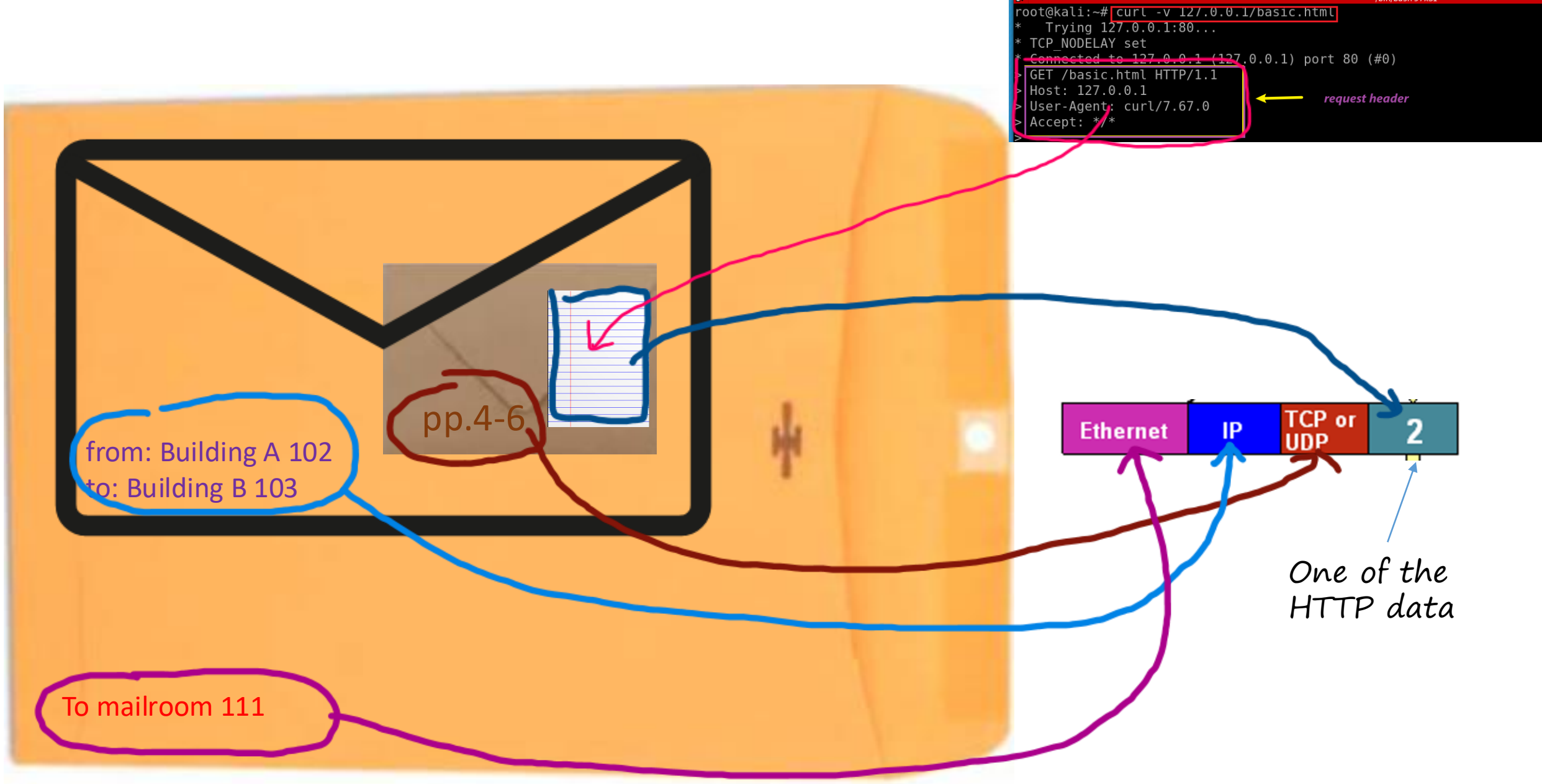
from: Building A 102
to: Building B 103

**Rules**:

1. All pages must be put in envelopes
2. each envelop max 3 pages
3. All internal mail must be delivered to the mail room
4. Internal emails delivered are done by staff, not an employee themselves





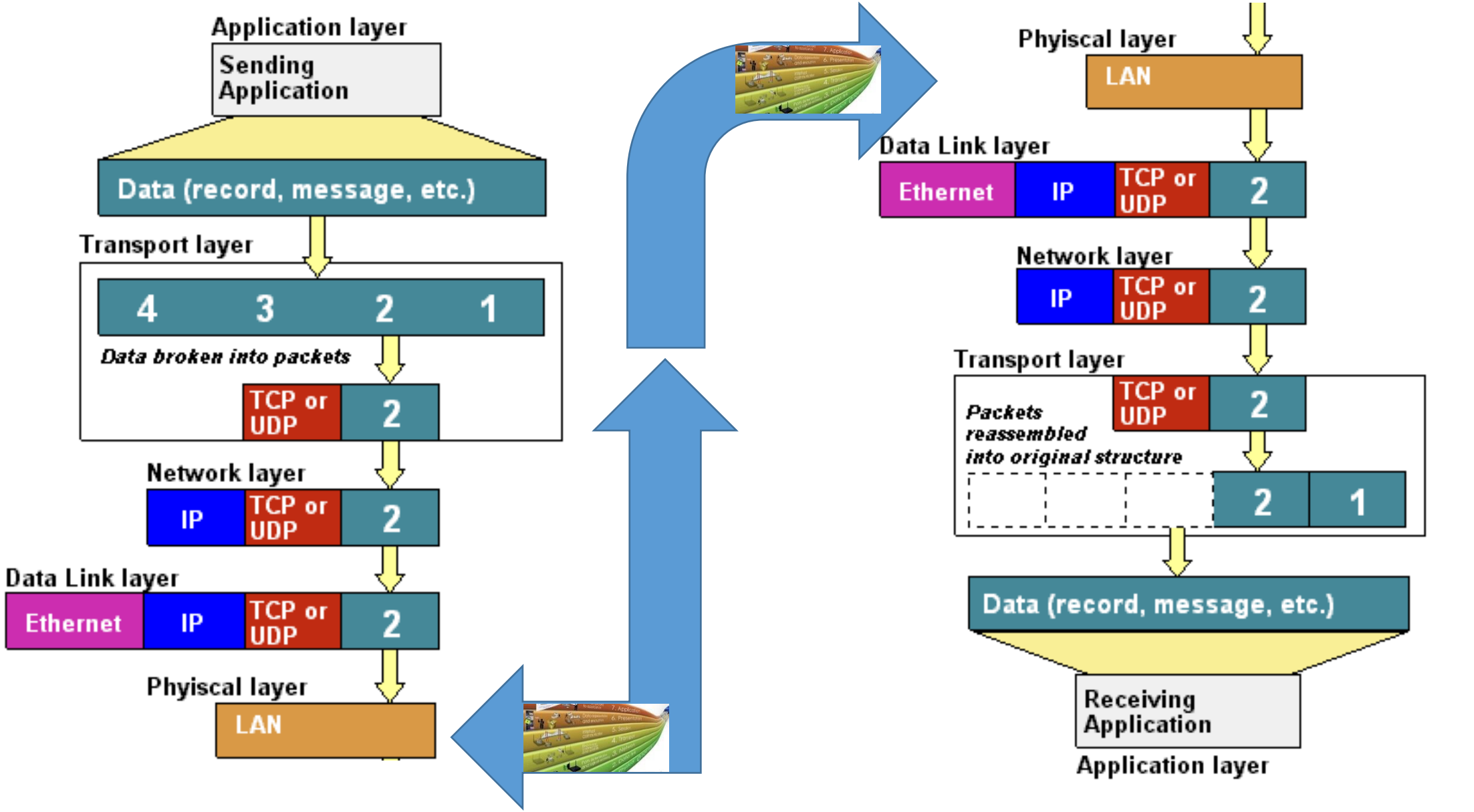
Data

Segment

Packet

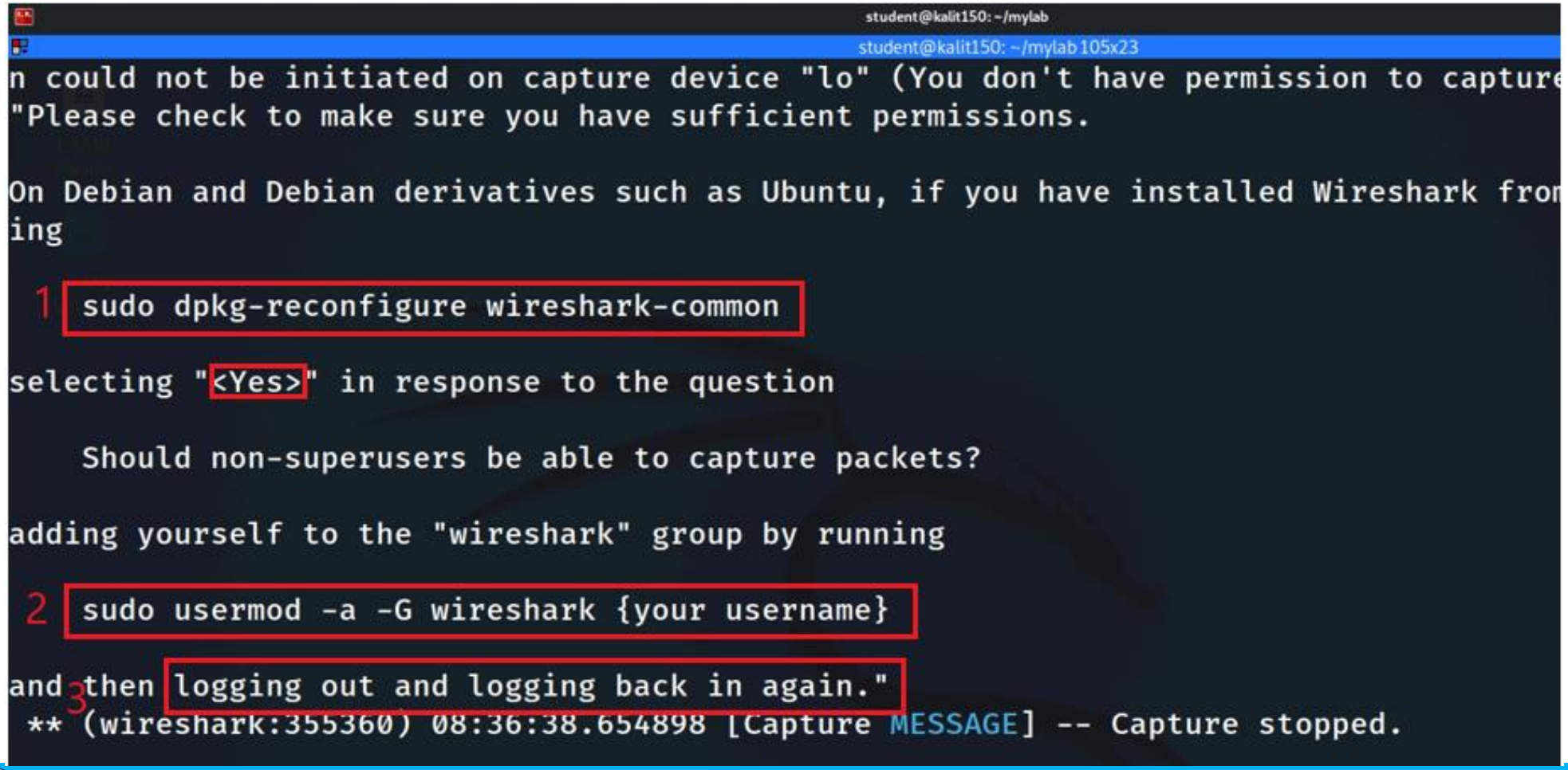
Frame

bit

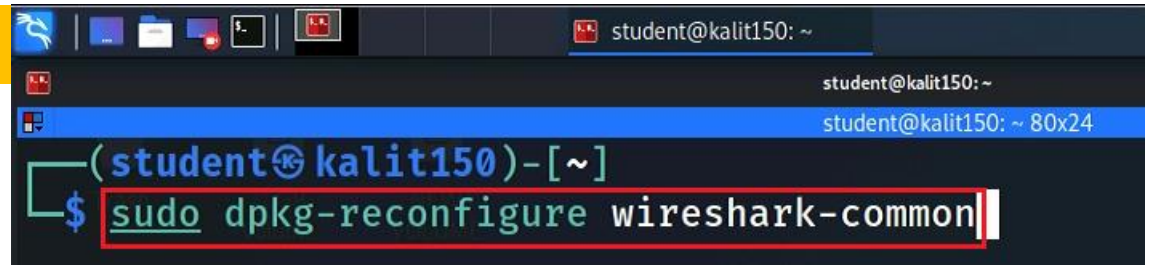


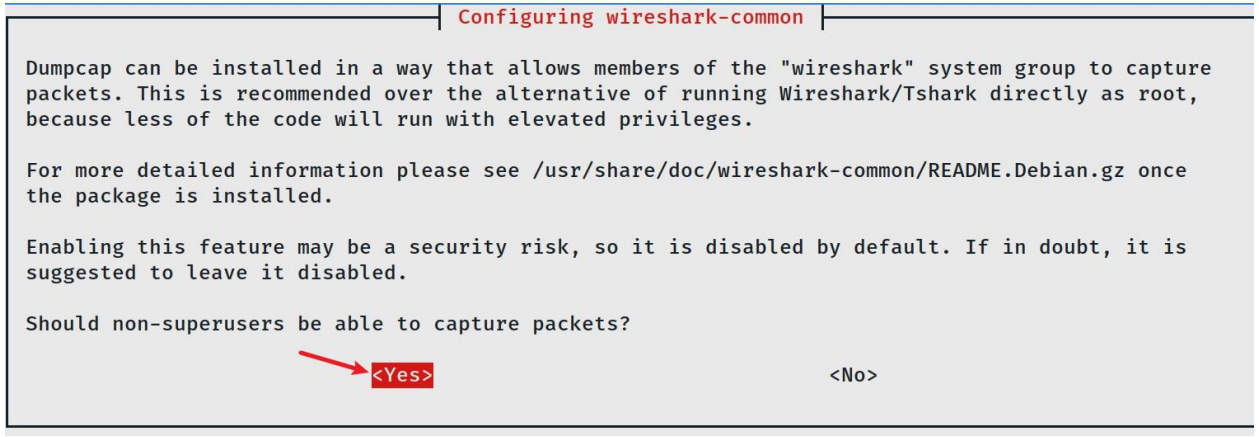
# Capture HTTP traffic using Wireshark

# Reconfigure wireshark

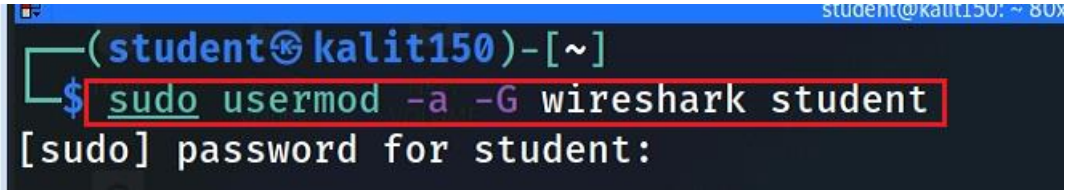


Reconfigure

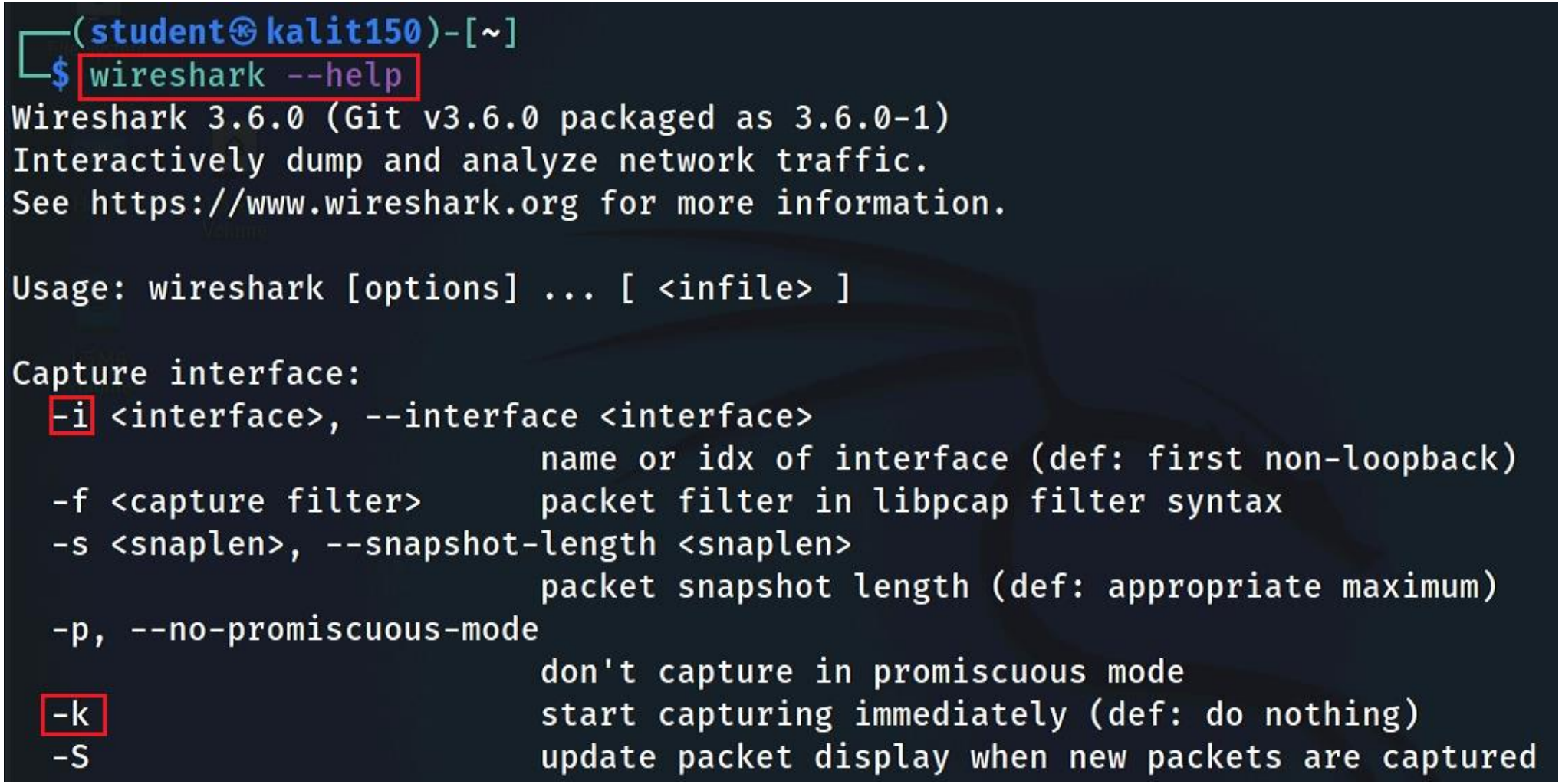



Add student to wireshark group




Logout and log back in



You don't need to have root to capture traffic

Clear browser cache

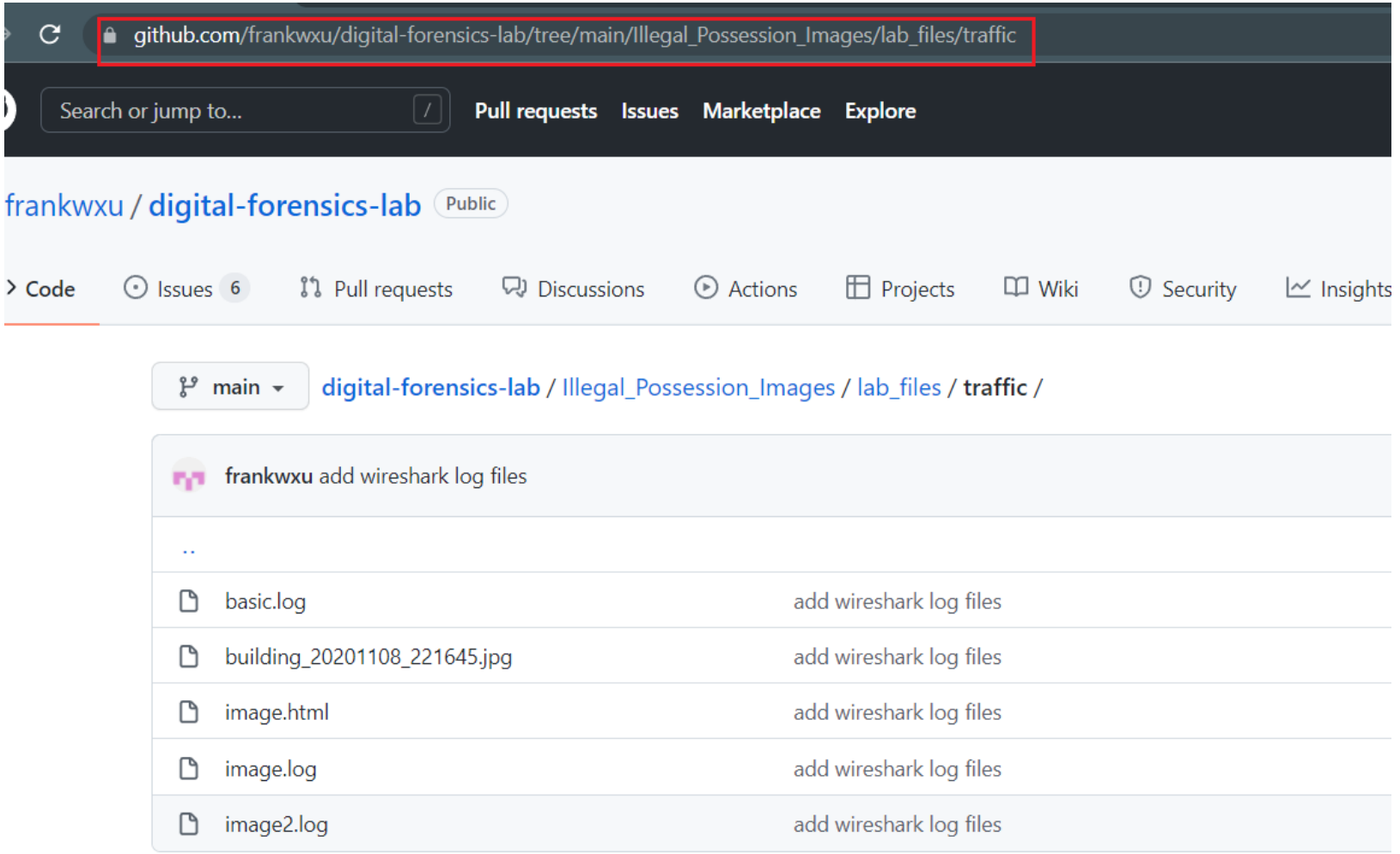
```
/bin/bash 102x1
root@kali:~#
root@kali:~# rm -r  ~/.cache/mozilla/firefox/*default*
root@kali:~#
```

```
/bin/bash 110x7
root@kali:~#
root@kali:~# mkdir traffic    1
root@kali:~#
root@kali:~# wireshark -i lo -k -w ~/traffic/basic.log    2
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLA
root@kali:~#    3. minimize wireshark    5. close wireshark
```

```
/bin/bash 110x22
root@kali:~# curl 127.0.0.1/basic.html    4
<!DOCTYPE html>
<html>
<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>
root@kali:~#
```

change interface -i lo to eth0 if you need to capture live traffic

# You can download the recaptured traffic




command to download the `basic.log` file:

*wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/basic.log*


```
root@kali:~# wireshark ~/traffic/basic.log
```

basic.log

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.000000000 | 127.0.0.1 | 127.0.0.1 | TCP | 74 | 57338 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PE |
| 2 | 0.000008171 | 127.0.0.1 | 127.0.0.1 | TCP | 74 | 80 → 57338 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65 |
| 3 | 0.000014783 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 57338 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=41024 |
| 4 | 0.000035452 | 127.0.0.1 | 127.0.0.1 | HTTP | 149 | GET /basic.html HTTP/1.1 |
| 5 | 0.000042870 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408 Len=0 TSval=410 |
| 6 | 0.000160688 | 127.0.0.1 | 127.0.0.1 | HTTP | 418 | HTTP/1.1 200 OK (text/html) |
| 7 | 0.000202191 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280 Len=0 TSval=41 |
| 8 | 0.000307996 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 57338 → 80 [FIN, ACK] Seq=84 Ack=353 Win=65536 Len=0 TSv |
| 9 | 0.000321475 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 80 → 57338 [FIN, ACK] Seq=353 Ack=85 Win=65536 Len=0 TSv |
| 10 | 0.000323997 | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 57338 → 80 [ACK] Seq=85 Ack=354 Win=65536 Len=0 TSval=41 |

handshake

http request

http response

tear down

wget https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/traffic/basic.log

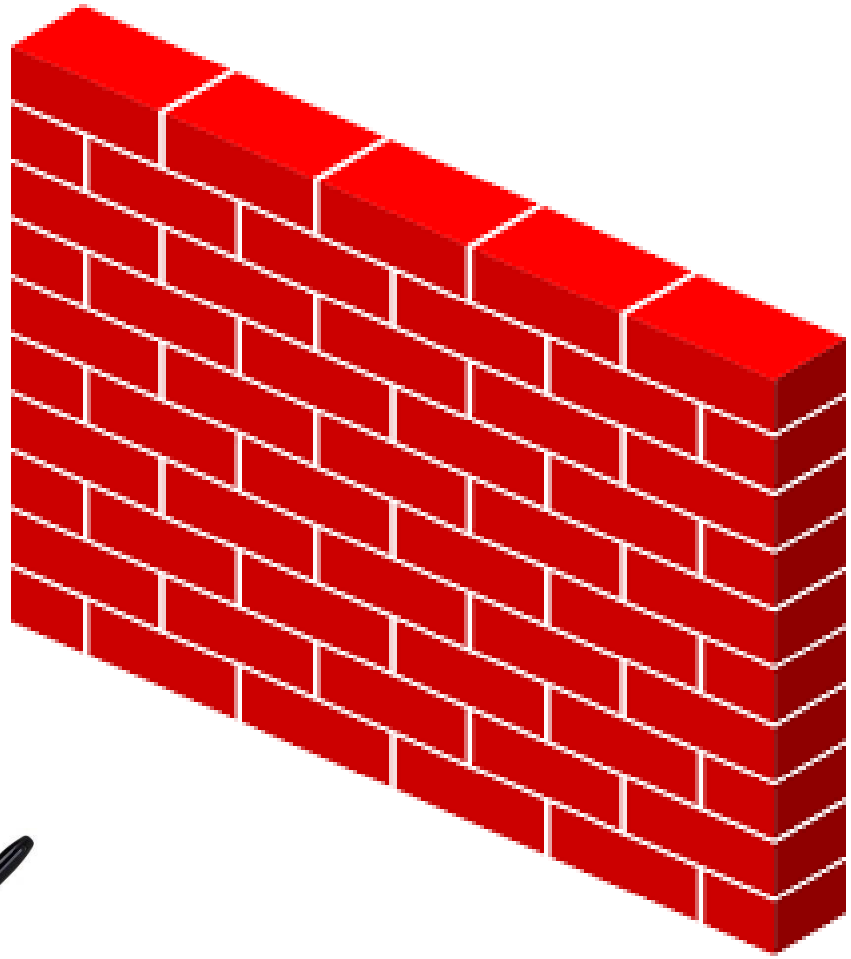
# Three-way handshake

No. 1, 2, and 3

# The purposes of handshake

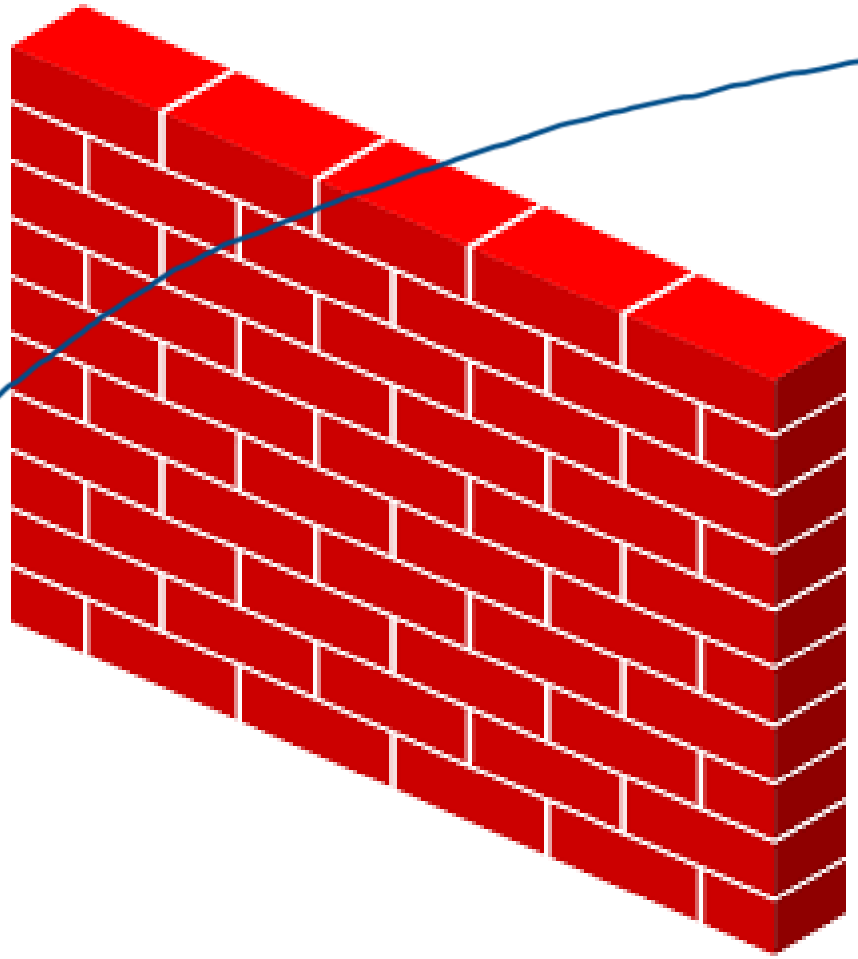
- Two participants establish a communication link at the start of the communication
  - Two parties are named as “Alice and Bob”
  - Before full communication begins
  - Make sure both sides ready to communicate
- Each party picks and exchanges a random number
  - as initial reference ID (byte) to keep track of each byte sending and receiving
    - e.g., Bob 6 and Alice 141
  - receiver expects ID after three messages


# ARE YOU READY ?

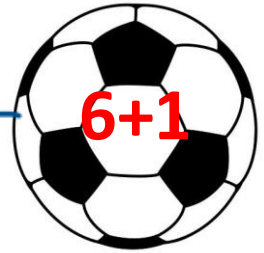


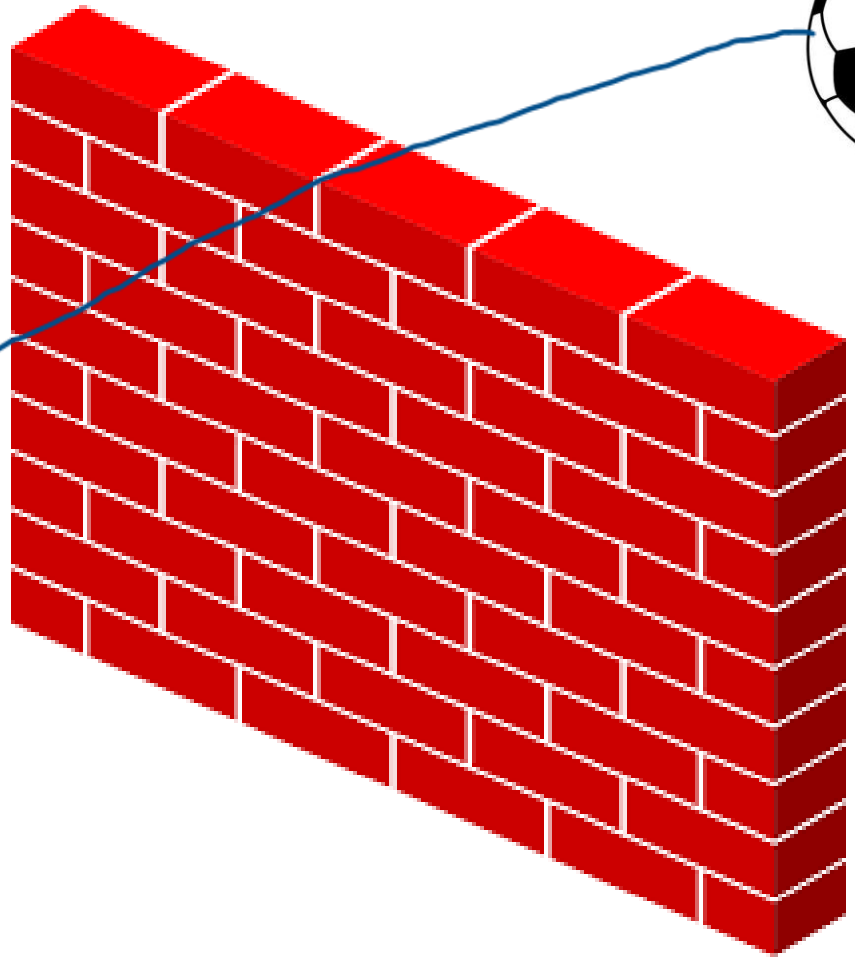
exchange reference ID

# ARE YOU READY ?




# ARE YOU READY ?

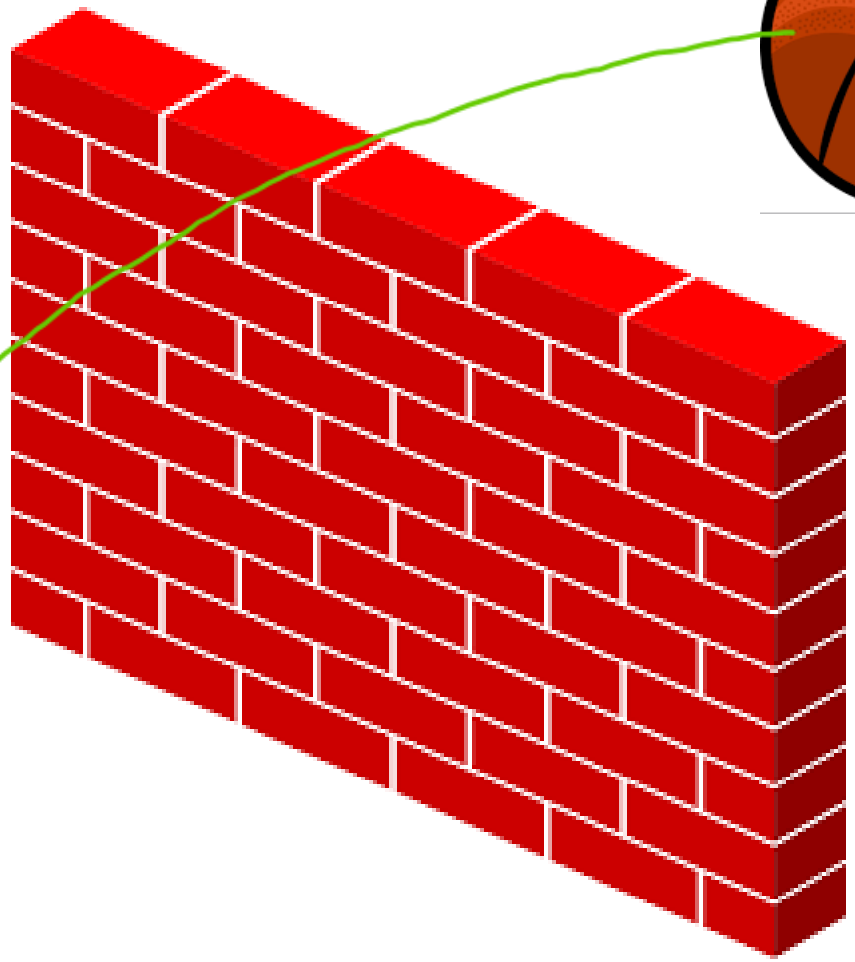


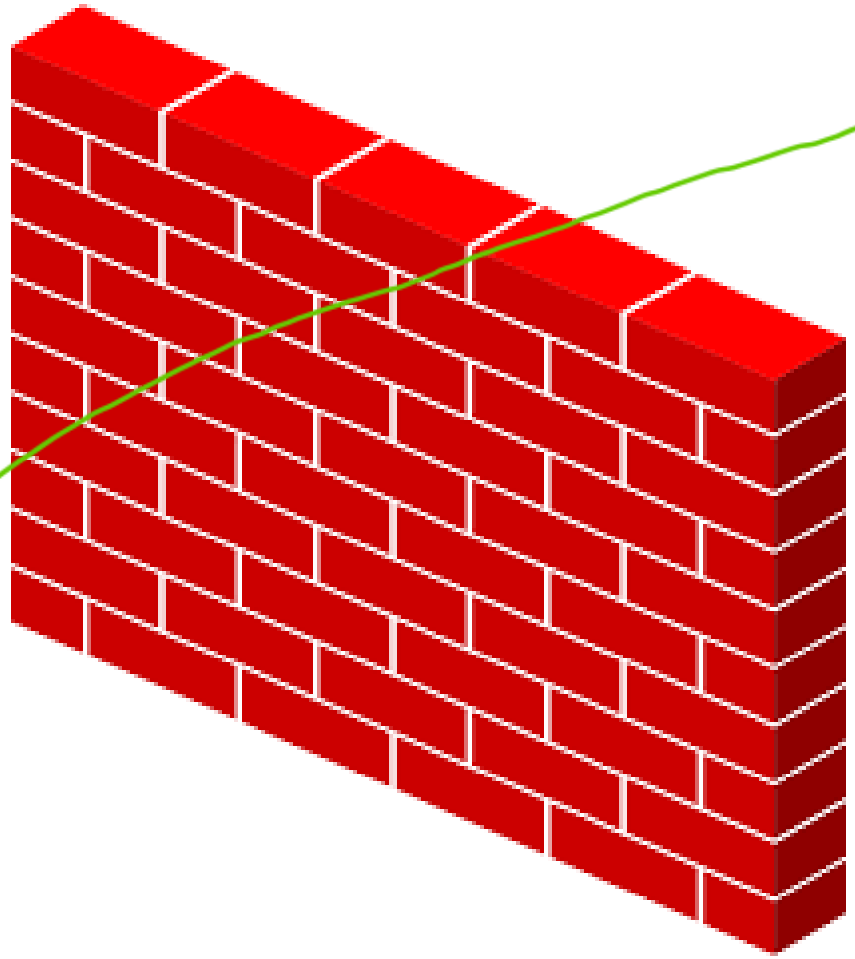
Reply:

- I have received the ball labeled with # 6.
- If you send me a message (contains multiple packets), I expect the first byte should be labeled as 6+1

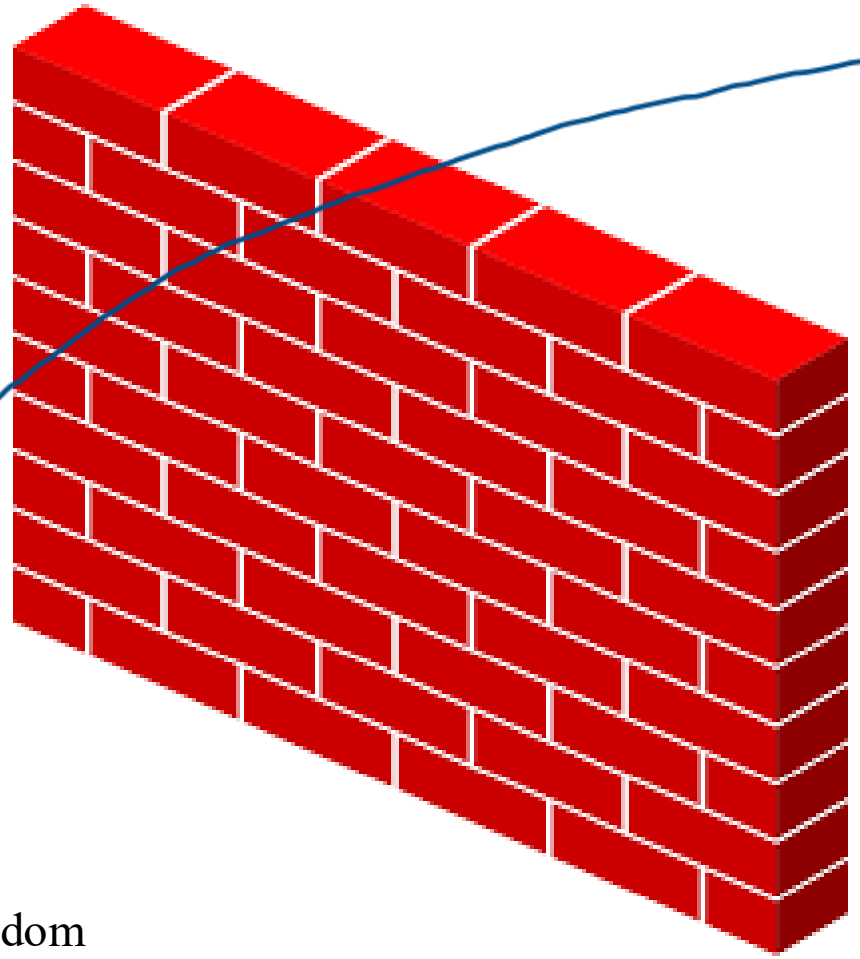

# ARE YOU READY ?

# ARE YOU READY ?



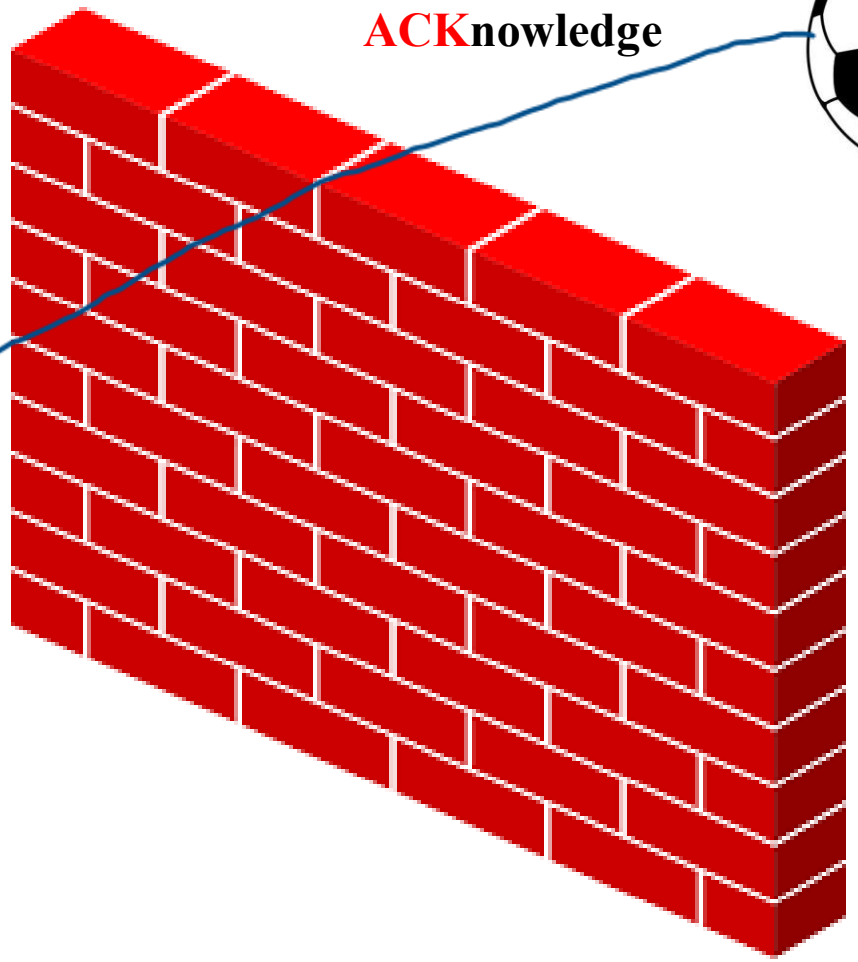

# ARE YOU READY ?



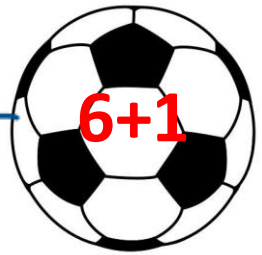
SYN message


**Sequence #:** random

# ARE YOU READY ?



ACKnowledge

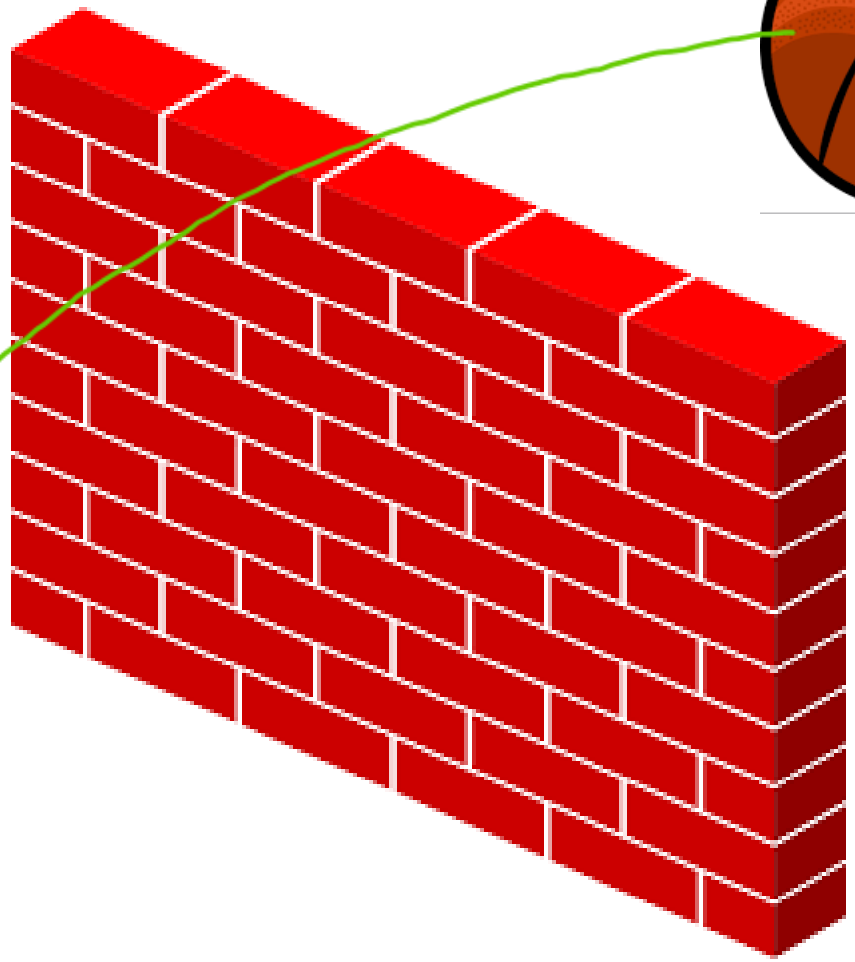



**ACKnowledge with seq#**

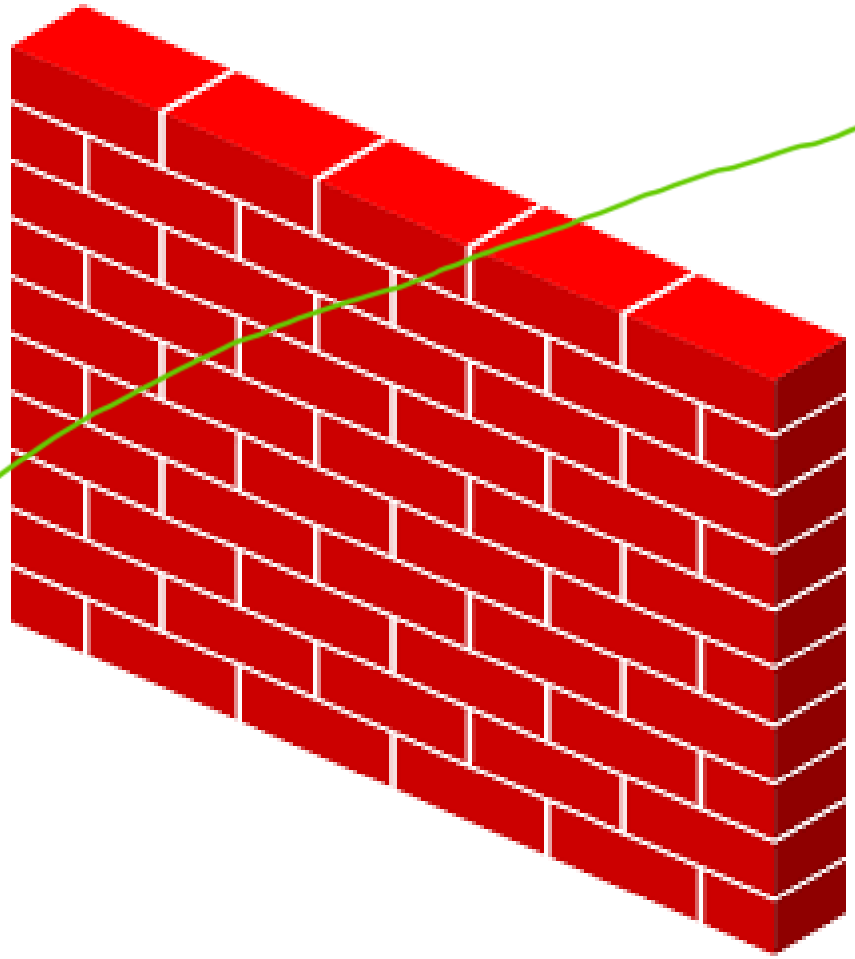
- I've received your request with seq # 6
- I expect to receive your next seq# 6+ 1

# ARE YOU READY ?

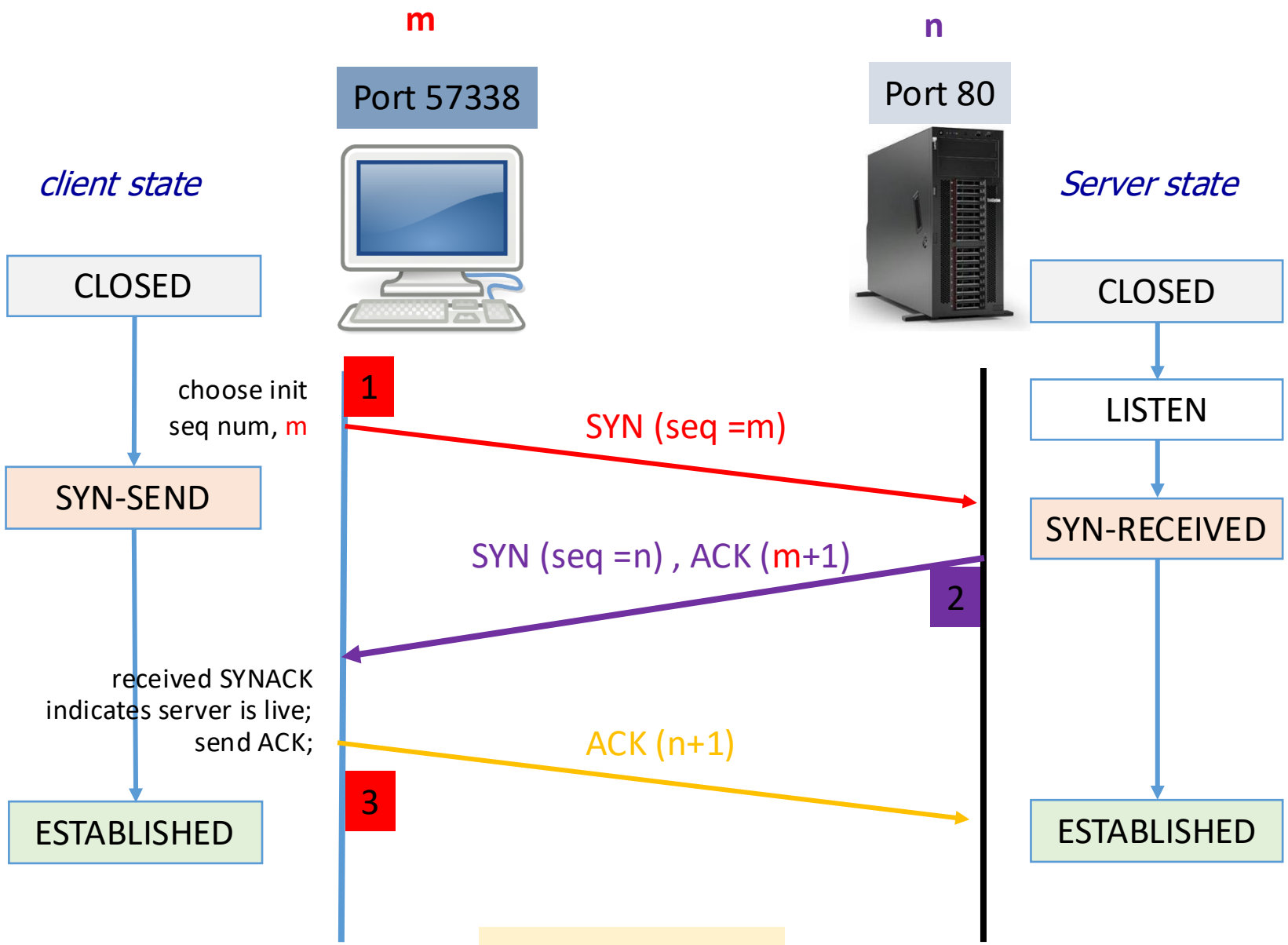
SYN message

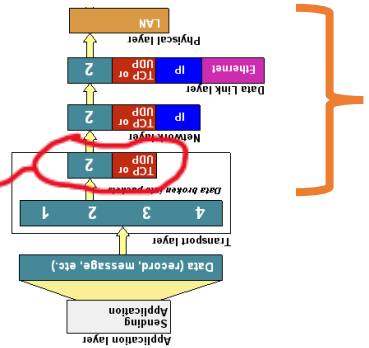
# ARE YOU READY ?



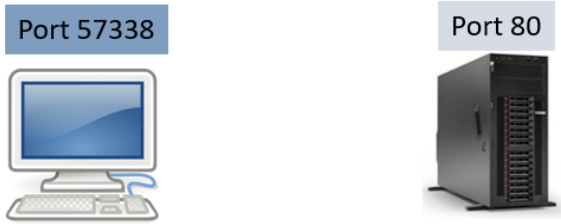
ACKnowledge

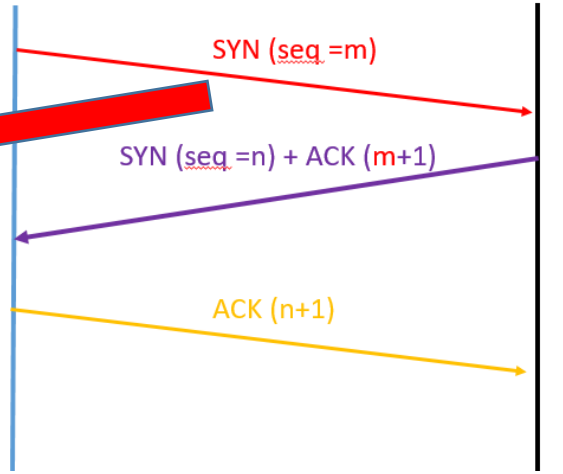




- **Sequence #:**
  - the byte number of the **first** byte of data in the TCP segment **sent**
  - beginning at random # or 0 (relative seq#)
- **ACKnowledge #**: the sequence number of the **next** byte the receiver **expects** to **receive**.
  - Seq # + size of packet + 1
  - The receiver ack'ing sequence number **x** acknowledges receipt of all data bytes less than (but not including) byte number **x**
  - +1 for SYN



Layers involved in handshaking




1

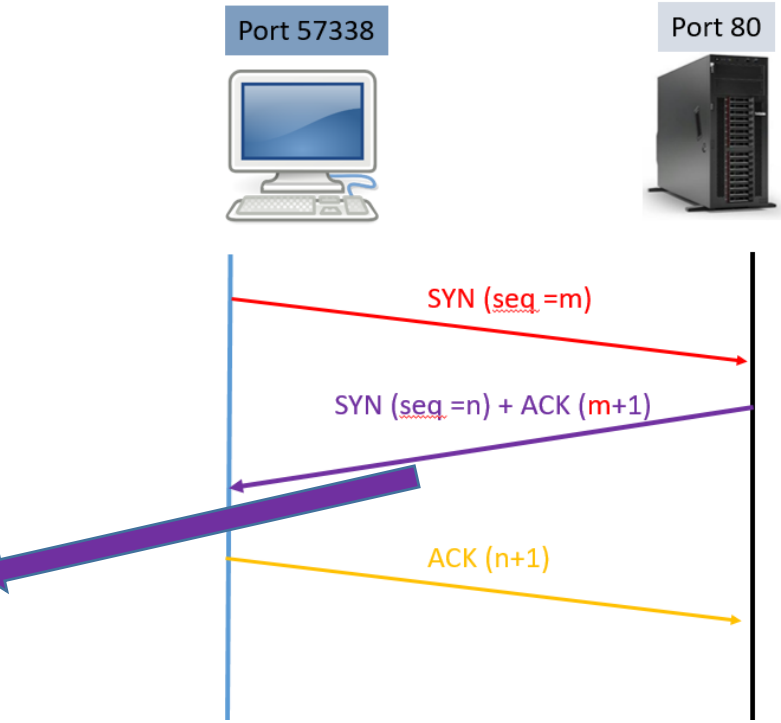
| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 1 | 0.00000… | 127.0.0.1 | 127.0.0.1 | TCP | 74 | 57338 → 80 [SYN] Seq=0 |
| 2 | 0.00000… | 127.0.0.1 | 127.0.0.1 | TCP | 74 | 80 → 57338 [SYN, ACK] S |
| 3 | 0.00001… | 127.0.0.1 | 127.0.0.1 | TCP | 66 | 57338 → 80 [ACK] Seq=1 |

```
▸ Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 57338, Dst Port: 80, Seq: 0,
    Source Port: 57338
    Destination Port: 80
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 0    (relative sequence number)
    Sequence number (raw): 2637494915
    [Next sequence number: 1    (relative sequence number)]
    Acknowledgment number: 0
    Acknowledgment number (raw): 0
    1010 .... = Header Length: 40 bytes (10)
  ▸ Flags: 0x002 (SYN)
```

m

**Client**:

- I am sending seq # **m**
- I will use the seq # **m** to indicate the byte number of the first byte of data in the TCP segment sent



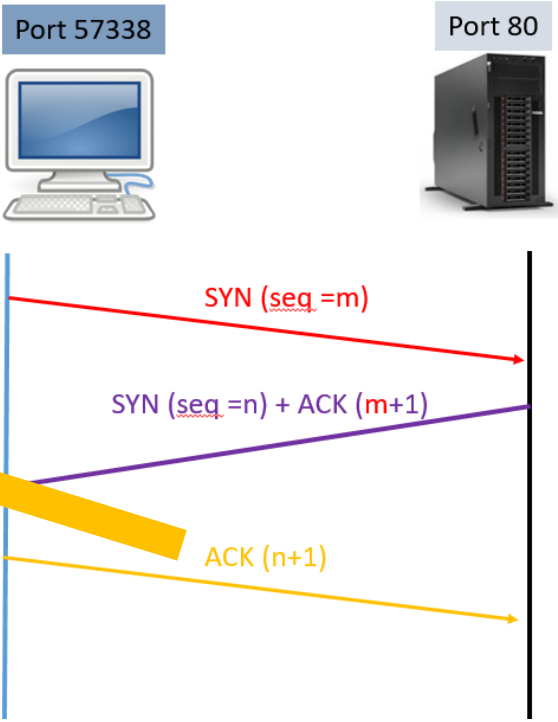

```
    1 0.00000… 127.0.0.1  127.0.0.1  TCP      74 57338 → 80 [SYN] Seq=0 Win=65495
2   2 0.00000… 127.0.0.1  127.0.0.1  TCP      74 80 → 57338 [SYN, ACK] Seq=0 Ack=1
    3 0.00001… 127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=1 Ack=1 Win=
▸ Frame 2: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 80, Dst Port: 57338, Seq: 0, Ack: 1, Len
    Source Port: 80
    Destination Port: 57338
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 0    (relative sequence number)
    Sequence number (raw): 335975064
    [Next sequence number: 1    (relative sequence number)]
    Acknowledgment number: 1    (relative ack number)
    Acknowledgment number (raw): 2637494916
    1010 .... = Header Length: 40 bytes (10)
  ▸ Flags: 0x012 (SYN, ACK)
```

n

m+ 1

**Server**:

- I've received your seq # m
- I expect to receive your next seq# m+ 1
- I'm sending seq # n and I will use it to indicate the byte number of the first byte of data in the TCP packet I am sending




```
    1 0.00000… 127.0.0.1  127.0.0.1  TCP      74 57338 → 80 [SYN] Seq=0 Win=65495 L
    2 0.00000… 127.0.0.1  127.0.0.1  TCP      74 80 → 57338 [SYN, ACK] Seq=0 Ack=1
3   3 0.00001… 127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=1 Ack=1 Win=6

▸ Frame 3: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface l
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 57338, Dst Port: 80, Seq: 1, Ack: 1, Len
    Source Port: 57338
    Destination Port: 80
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 1    (relative sequence number)
    Sequence number (raw): 2637494916
    [Next sequence number: 1    (relative sequence number)]
    Acknowledgment number: 1    (relative ack number)
    Acknowledgment number (raw): 335975065
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x010 (ACK)
    Window size value: 512
```

**m+ 1**

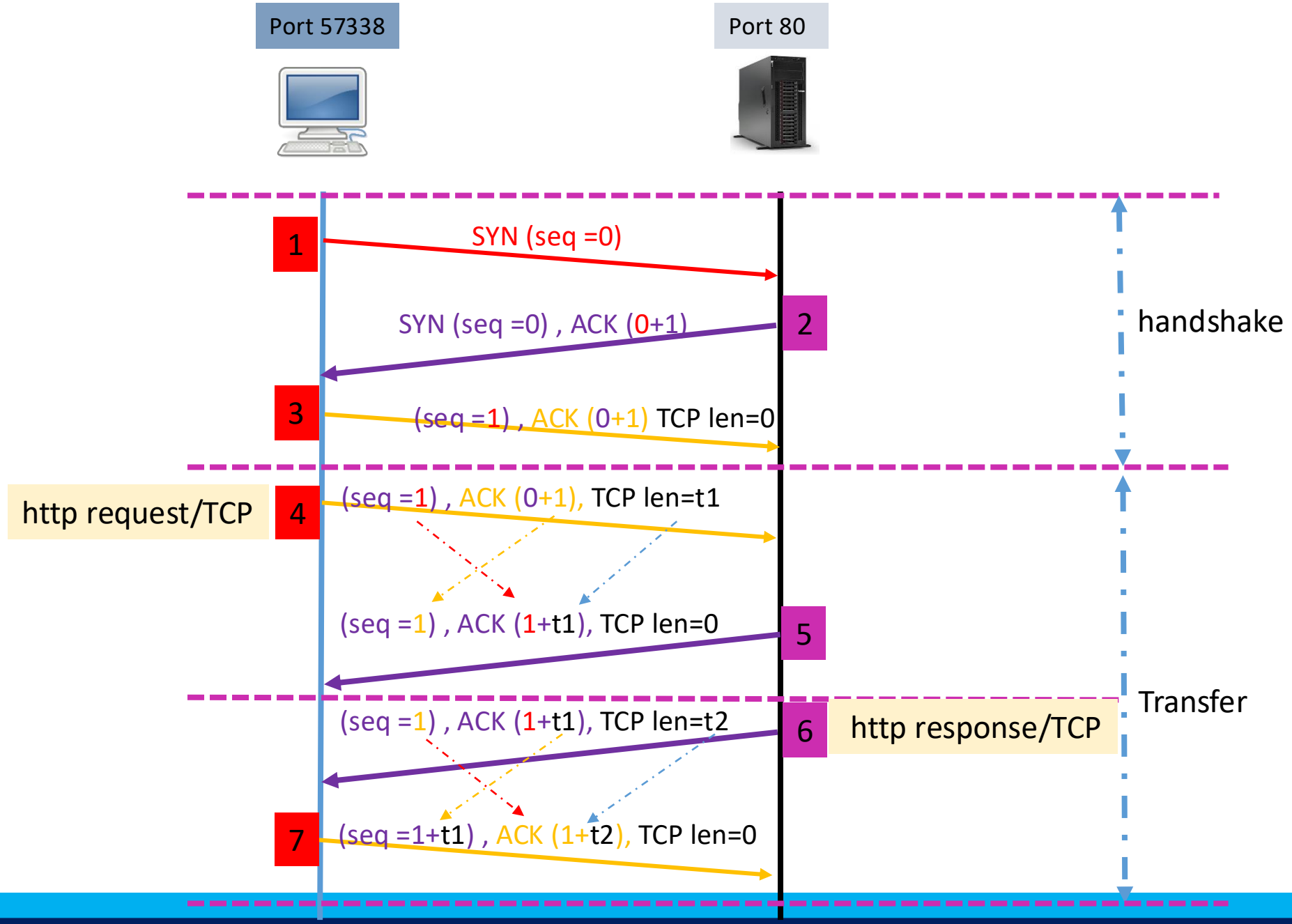
**n +1**

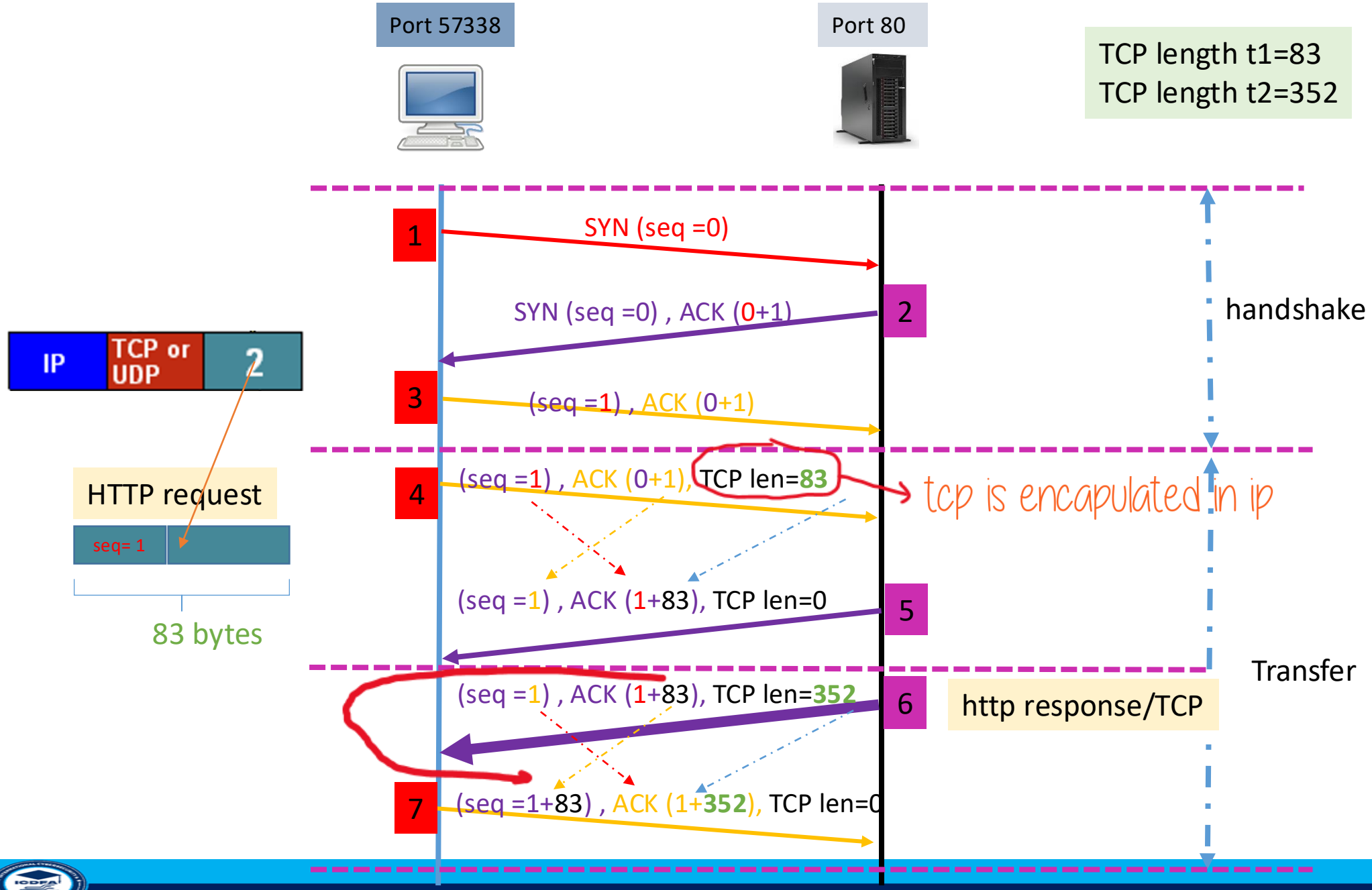
**Client**:

- OK. I am sending seq # **m+ 1** as you requested
- I ack that I have received **n** and
- I expect to receive package starts with seq # **n +1**

# HTTP request protocol

No. 4





## Show http request

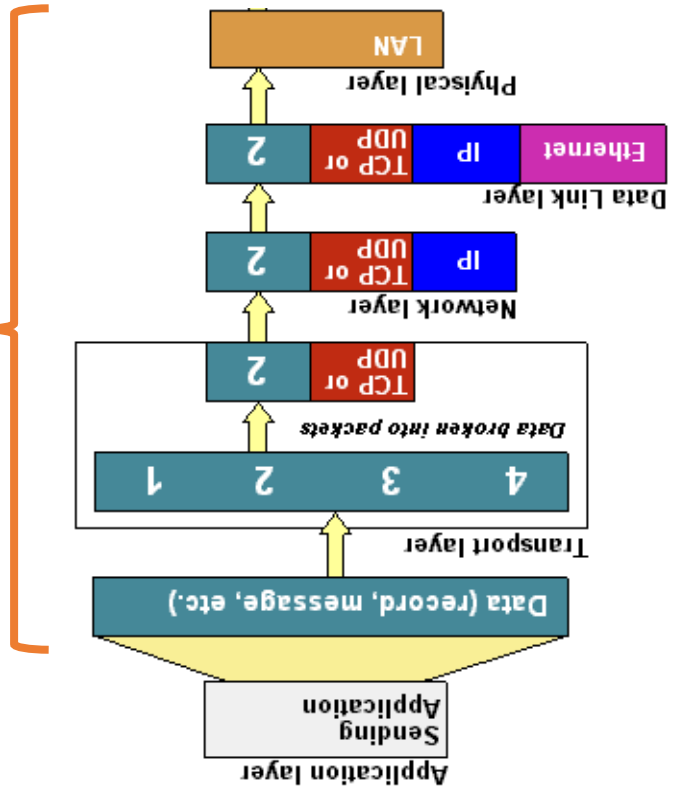
```
3 0 000014…  127.0.0.1   127.0.0.1   TCP      66 57338 → 80 [ACK] Seq=1 Ack=1 Win=65536
4   000035…  127.0.0.1   127.0.0.1   HTTP    149 GET /basic.html HTTP/1.1
5   000042…  127.0.0.1   127.0.0.1   TCP      66 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408
```

4 1

```
▸ Frame 4: 149 bytes on wire (1192 bits), 149 bytes captured (1192 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▸ Transmission Control Protocol, Src Port: 57338, Dst Port: 80, Seq: 1, Ack: 1, Len: 83
▾ Hypertext Transfer Protocol                                   2
  ▾ GET /basic.html HTTP/1.1\r\n                                3
    ▸ [Expert Info (Chat/Sequence): GET /basic.html HTTP/1.1\r\n]
      Request Method: GET                                       4
      Request URI: /basic.html
      Request Version: HTTP/1.1
    Host: 127.0.0.1\r\n
    User-Agent: curl/7.67.0\r\n                                 5
    Accept: */*\r\n
    \r\n
    [Full request URI: http://127.0.0.1/basic.html]
    [HTTP request 1/1]
    [Response in frame: 6]
```

```
0000  00 00 00 00 00 00 00 00  00 00 00 00 08 00 45 00   ·····E·
0010  00 87 77 77 40 00 40 06  c4 f7 7f 00 00 01 7f 00   ··ww@·@·
0020  00 01 df fa 00 50 9d 34  fa 84 14 06 92 99 80 18   ·····P·4
0030  02 00 fe 7b 00 00 01 01  08 0a f4 85 ac a7 f4 85   ···{····
0040  ac a7 47 45 54 20 2f 62  61 73 69 63 2e 68 74 6d   ··GET /b asic.htm
0050  6c 20 48 54 54 50 2f 31  2e 31 0d 0a 48 6f 73 74   l HTTP/1 .1··Host
0060  3a 20 31 32 37 2e 30 2e  30 2e 31 0d 0a 55 73 65   : 127.0. 0.1··Use
0070  72 2d 41 67 65 6e 74 3a  20 63 75 72 6c 2f 37 2e   r-Agent:  curl/7.
0080  36 37 2e 30 0d 0a 41 63  63 65 70 74 3a 20 2a 2f   67.0··Ac cept: */
0090  2a 0d 0a 0d 0a                                     *····
```

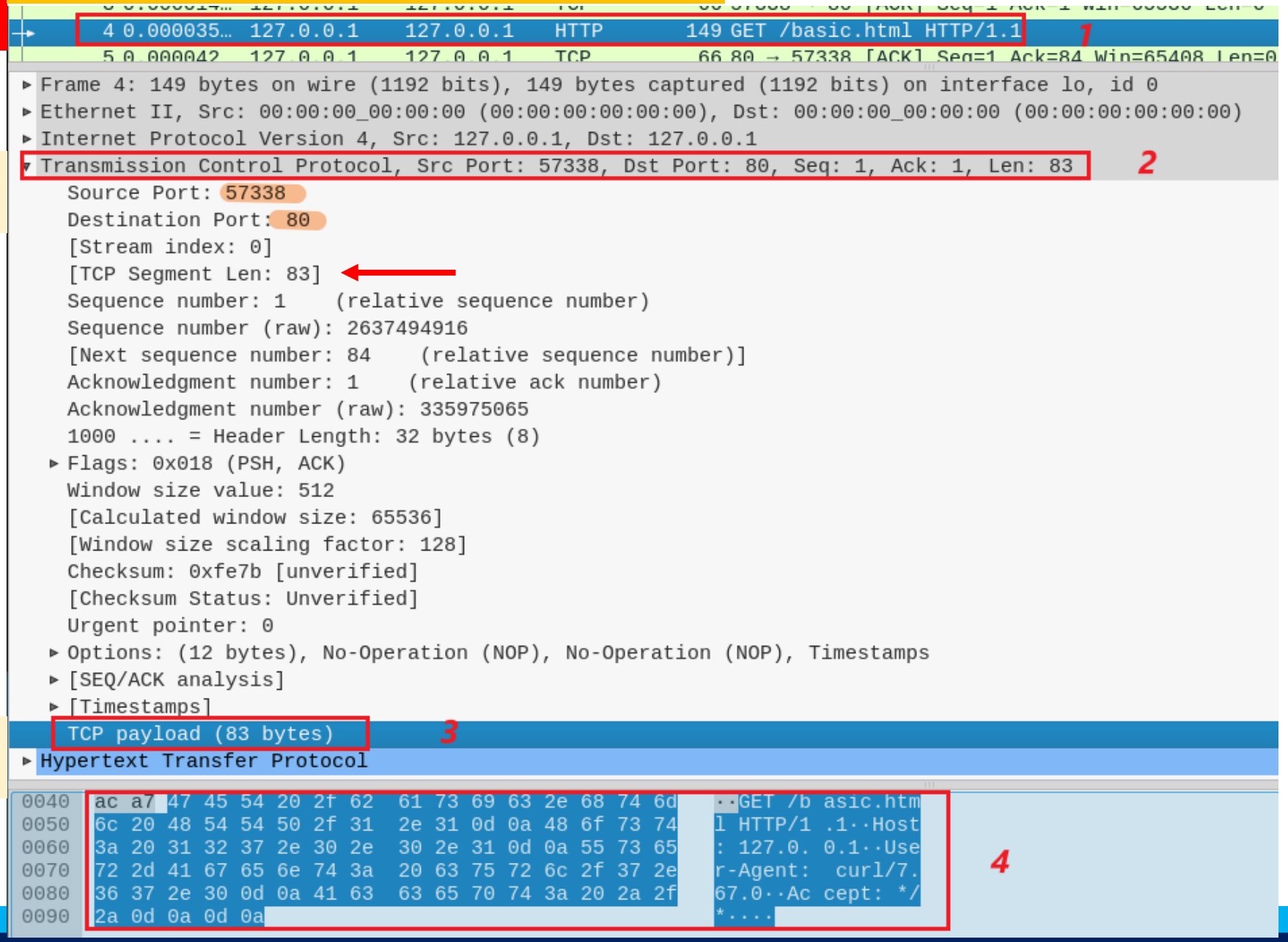
6




Layers involved in http request

Show http request

4

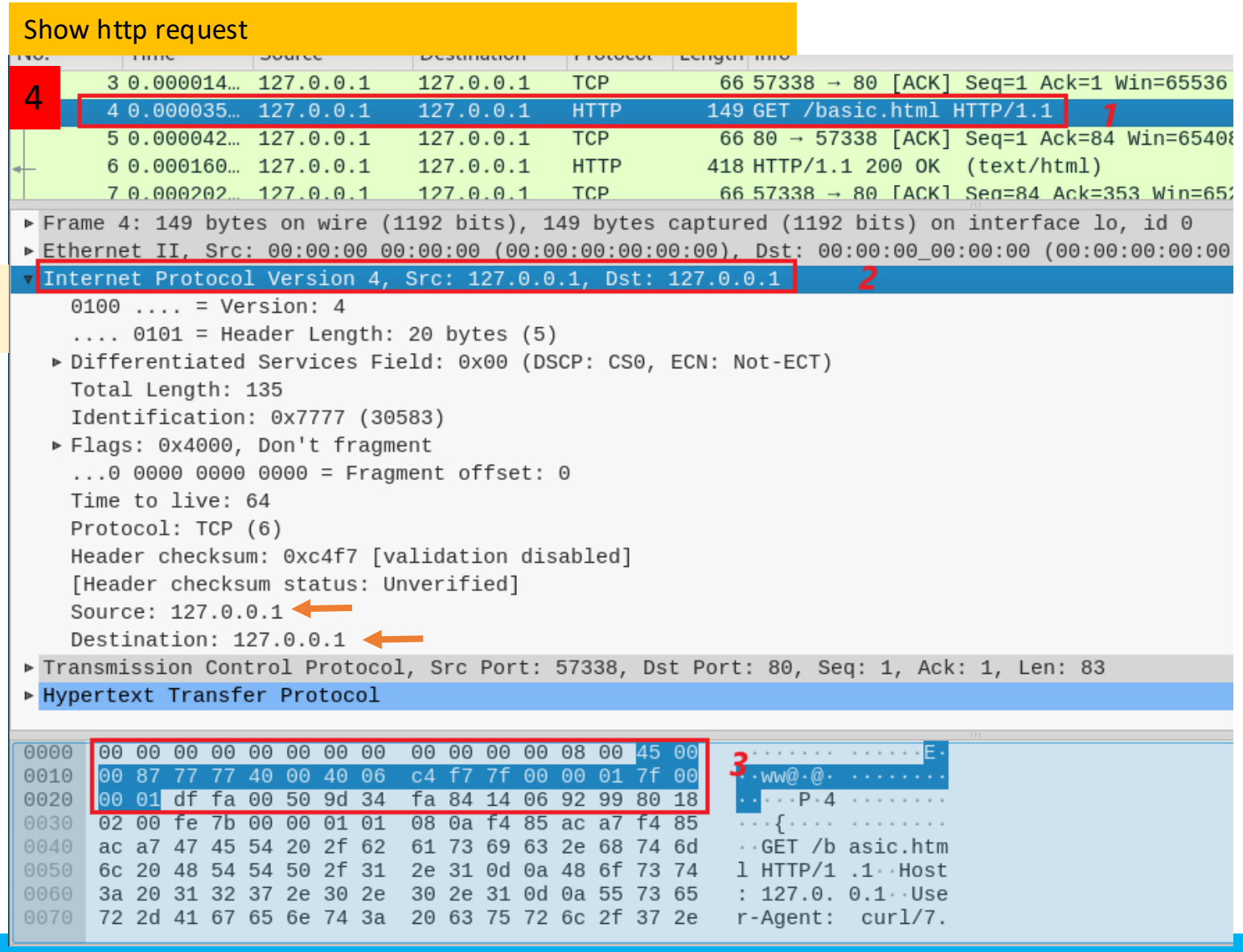



Show TCP segment 4

Show TCP payload

Show http request

Show IP packet 4



Show http request

4

```
3 0.000014… 127.0.0.1   127.0.0.1   TCP      66 57338 → 80 [ACK] Seq=1 Ack=1 Win=65
4 0.000035… 127.0.0.1   127.0.0.1   HTTP    149 GET /basic.html HTTP/1.1
5 0.000042… 127.0.0.1   127.0.0.1   TCP      66 80 → 57338 [ACK] Seq=1 Ack=84 Win=6
6 0.000160… 127.0.0.1   127.0.0.1   HTTP    418 HTTP/1.1 200 OK  (text/html)
```

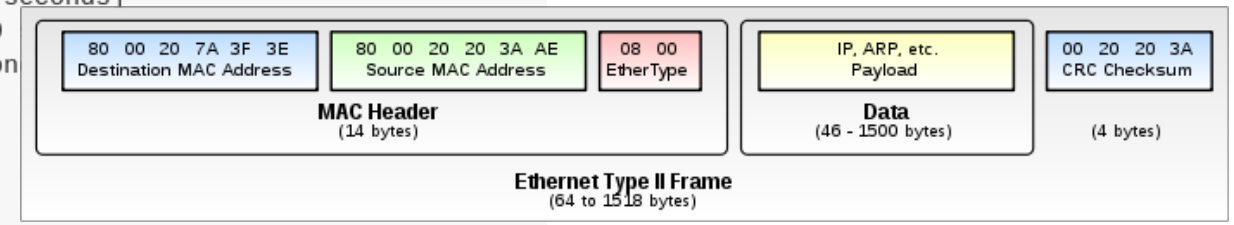
1

Ethernet Frame 4

```
▾ Frame 4: 149 bytes on wire (1192 bits), 149 bytes captured (1192 bits) on interface lo, id 0
  ▾ Interface id: 0 (lo)
      Interface name: lo
    Encapsulation type: Ethernet (1)
    Arrival Time: Oct 27, 2020 23:18:22.852038229 EDT
    [Time shift for this packet: 0.000000000 seconds]
    Epoch Time: 1603855102.852038229 seconds
    [Time delta from previous captured frame: 0.000020669 seconds]
    [Time delta from previous displayed frame: 0.000020669
    [Time since reference or first frame: 0.000035452 secon
    Frame Number: 4
    Frame Length: 149 bytes (1192 bits)
    Capture Length: 149 bytes (1192 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:tcp:http]
    [Coloring Rule Name: HTTP]
    [Coloring Rule String: http || tcp.port == 80 || http2]
```

2

3




MAC address of lo

Show Ethernet 802.3

```
▾ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (
  ▸ Destination: 00:00:00_00:00:00 (00:00:00:00:00:00)
  ▸ Source: 00:00:00_00:00:00 (00:00:00:00:00:00)
    Type: IPv4
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
```

4

ARP (0x0806)

```
0000  00 00 00 00 00 00 00 00  00 00 00 00 08 00 45 00   ........ ......E·
0010  00 87 77 77 40 00 40 06  c4 f7 7f 00 00 01 7f 00   ··ww@·@· ........
0020  00 01 df fa 00 50 9d 34  fa 84 14 06 92 99 80 18   ·····P·4 ........
0030  02 00 fe 7b 00 00 01 01  08 0a f4 85 ac a7 f4 85   ···{···· ........
0040  ac a7 47 45 54 20 2f 62  61 73 69 63 2e 68 74 6d   ··GET /b asic.htm
0050  6c 20 48 54 54 50 2f 31  2e 31 0d 0a 48 6f 73 74   l HTTP/1 .1··Host
0060  3a 20 31 32 37 2e 30 2e  30 2e 31 0d 0a 55 73 65   : 127.0. 0.1··Use
0070  72 2d 41 67 65 6e 74 3a  20 63 75 72 6c 2f 37 2e   r-Agent:  curl/7.
```

5

lo interface is not associated with a hardware network interface (it's a virtual loopback interface), it does not have an Ethernet hardware address (MAC address).

Server ACKnowledges the request

5

```
    4 0.00003… 127.0.0.1  127.0.0.1 HTTP     149 GET /basic.html HTTP/1.1
    5 0.00004… 127.0.0.1  127.0.0.1 TCP       66 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408 Le
    6 0.00016… 127.0.0.1  127.0.0.1 HTTP     418 HTTP/1.1 200 OK  (text/html)
    7 0.00020… 127.0.0.1  127.0.0.1 TCP       66 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280
```

1

```
▸ Frame 5: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 80, Dst Port: 57338, Seq: 1, Ack: 84, Len: 0
    Source Port: 80
    Destination Port: 57338
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 1    (relative sequence number)
    Sequence number (raw): 335975065
    [Next sequence number: 1    (relative sequence number)]
    Acknowledgment number: 84    (relative ack number)
    Acknowledgment number (raw): 2637494999
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x010 (ACK)
    Window size value: 511
```

2

ACK 84 =1+ TCP length

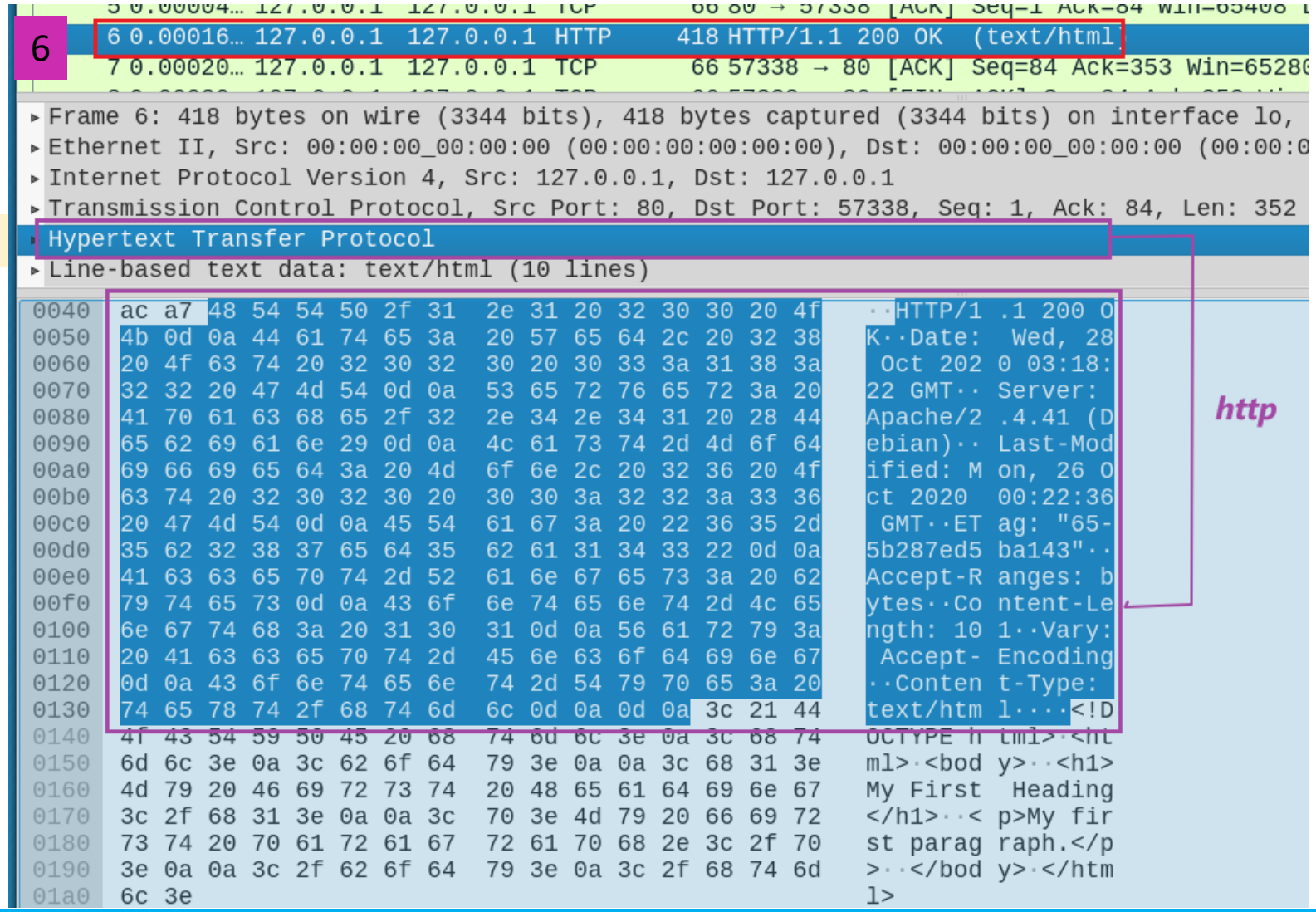
```
0000  00 00 00 00 00 00 00 00  00 00 00 00 08 00 45 00   ........ ......E·
0010  00 34 71 a9 40 00 40 06  cb 18 7f 00 00 01 7f 00   ·4q·@·@· ........
0020  00 01 00 50 df fa 14 06  92 99 9d 34 fa d7 80 10   ···P···· ···4····
0030  01 ff fe 28 00 00 01 01  08 0a f4 85 ac a7 f4 85   ···(···· ........
0040  ac a7                                             ··
```

TCP

# HTTP response protocol

No. 5 and 6

## Server send HTTP response head



## Server send HTTP response data (simple.html)

6

```
    5 0.00004… 127.0.0.1  127.0.0.1  TCP      66 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408 Len=0 TS
    6 0.00016… 127.0.0.1  127.0.0.1  HTTP    418 HTTP/1.1 200 OK  (text/html)
    7 0.00020… 127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280 Len=0
```

```
▸ Frame 6: 418 bytes on wire (3344 bits), 418 bytes captured (3344 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▸ Transmission Control Protocol, Src Port: 80, Dst Port: 57338, Seq: 1, Ack: 84, Len: 352
▸ Hypertext Transfer Protocol
▸ Line-based text data: text/html (10 lines)
```

simple.html

```
0040  ac a7 48 54 54 50 2f 31  2e 31 20 32 30 30 20 4f   ··HTTP/1 .1 200 O
0050  4b 0d 0a 44 61 74 65 3a  20 57 65 64 2c 20 32 38   K··Date:  Wed, 28
0060  20 4f 63 74 20 32 30 32  30 20 30 33 3a 31 38 3a    Oct 202 0 03:18:
0070  32 32 20 47 4d 54 0d 0a  53 65 72 76 65 72 3a 20   22 GMT·· Server:
0080  41 70 61 63 68 65 2f 32  2e 34 2e 34 31 20 28 44   Apache/2 .4.41 (D
0090  65 62 69 61 6e 29 0d 0a  4c 61 73 74 2d 4d 6f 64   ebian)·· Last-Mod
00a0  69 66 69 65 64 3a 20 4d  6f 6e 2c 20 32 36 20 4f   ified: M on, 26 O
00b0  63 74 20 32 30 32 30 20  30 30 3a 32 32 3a 33 36   ct 2020  00:22:36
00c0  20 47 4d 54 0d 0a 45 54  61 67 3a 20 22 36 35 2d    GMT··ET ag: "65-
00d0  35 62 32 38 37 65 64 35  62 61 31 34 33 22 0d 0a   5b287ed5 ba143"··
00e0  41 63 63 65 70 74 2d 52  61 6e 67 65 73 3a 20 62   Accept-R anges: b
00f0  79 74 65 73 0d 0a 43 6f  6e 74 65 6e 74 2d 4c 65   ytes··Co ntent-Le
0100  6e 67 74 68 3a 20 31 30  31 0d 0a 56 61 72 79 3a   ngth: 10 1··Vary:
0110  20 41 63 63 65 70 74 2d  45 6e 63 6f 64 69 6e 67    Accept- Encoding
0120  0d 0a 43 6f 6e 74 65 6e  74 2d 54 79 70 65 3a 20   ··Conten t-Type:
0130  74 65 78 74 2f 68 74 6d  6c 0d 0a 0d 0a 3c 21 44   text/htm l····<!D
0140  4f 43 54 59 50 45 20 68  74 6d 6c 3e 0a 3c 68 74   OCTYPE h tml>·<ht
0150  6d 6c 3e 0a 3c 62 6f 64  79 3e 0a 0a 3c 68 31 3e   ml>·<bod y>··<h1>
0160  4d 79 20 46 69 72 73 74  20 48 65 61 64 69 6e 67   My First  Heading
0170  3c 2f 68 31 3e 0a 0a 3c  70 3e 4d 79 20 66 69 72   </h1>··< p>My fir
0180  73 74 20 70 61 72 61 67  72 61 70 68 2e 3c 2f 70   st parag raph.</p
0190  3e 0a 0a 3c 2f 62 6f 64  79 3e 0a 3c 2f 68 74 6d   >··</bod y>·</htm
01a0  6c 3e                                              l>
```

*html*

## Server send HTTP response data (simple.html)

6

```
    5 0.00004… 127.0.0.1  127.0.0.1  TCP      66 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408 L
    6 0.00016… 127.0.0.1  127.0.0.1  HTTP    418 HTTP/1.1 200 OK  (text/html)
    7 0.00020… 127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280
```

TCP

```
▸ Frame 6: 418 bytes on wire (3344 bits), 418 bytes captured (3344 bits) on interface lo, i
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:0
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 80, Dst Port: 57338, Seq: 1, Ack: 84, Len: 352
    Source Port: 80
    Destination Port: 57338
    [Stream index: 0]
    [TCP Segment Len: 352]
    Sequence number: 1    (relative sequence number)
    Sequence number (raw): 335975065
    [Next sequence number: 353    (relative sequence number)]
    Acknowledgment number: 84    (relative ack number)
    Acknowledgment number (raw): 2637494999
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x018 (PSH, ACK)
    Window size value: 512
```

TCP

PSH: data must be transmitted promptly to the receiver

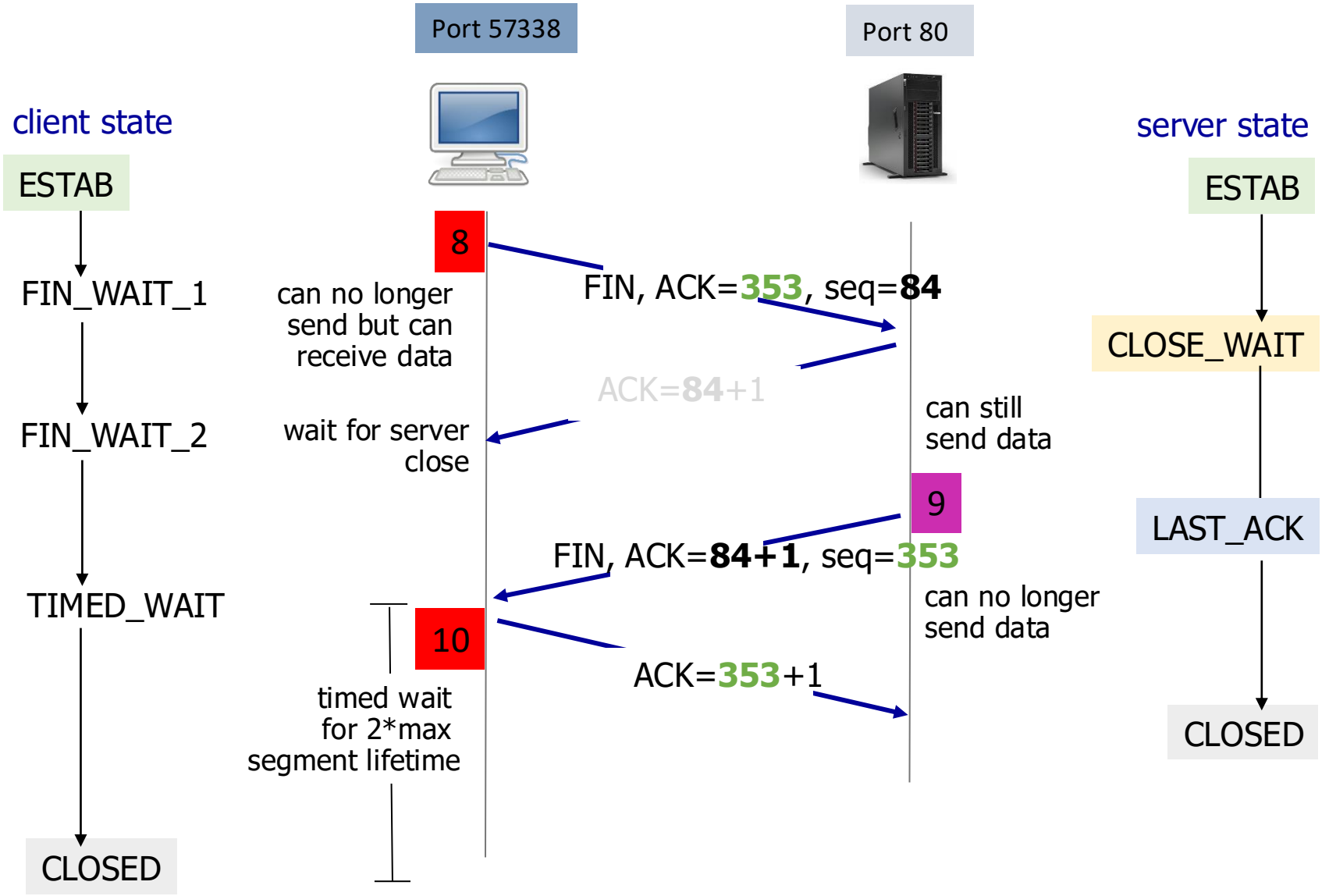
```
0020  00 01 00 50 df fa 14 06  92 99 9d 34 fa d7 80 18   ···P···· ···4····
0030  02 00 ff 88 00 00 01 01  08 0a f4 85 ac a8 f4 85   ........ ........
0040  ac a7 48 54 54 50 2f 31  2e 31 20 32 30 30 20 4f   ··HTTP/1 .1 200 O
0050  4b 0d 0a 44 61 74 65 3a  20 57 65 64 2c 20 32 38   K··Date:  Wed, 28
0060  20 4f 63 74 20 32 30 32  30 20 30 33 3a 31 38 3a    Oct 202 0 03:18:
0070  32 32 20 47 4d 54 0d 0a  53 65 72 76 65 72 3a 20   22 GMT·· Server:
0080  41 70 61 63 68 65 2f 32  2e 34 2e 34 31 20 28 44   Apache/2 .4.41 (D
0090  65 62 69 61 6e 29 0d 0a  4c 61 73 74 2d 4d 6f 64   ebian)·· Last-Mod
```

Client sends TCP acknowledgement

```
    5 0.00004… 127.0.0.1  127.0.0.1  TCP      66 80 → 57338 [ACK] Seq=1 Ack=84 Win=65408 Len=0 TS
    6 0.00016… 127.0.0.1  127.0.0.1  HTTP    418 HTTP/1.1 200 OK  (text/html)
7   7 0.00020… 127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280 Len=0
    8 0.00030… 127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [FIN, ACK] Seq=84 Ack=353 Win=65536 L
    9 0.00032… 127.0.0.1  127.0.0.1  TCP      66 80 → 57338 [FIN, ACK] Seq=353 Ack=85 Win=65536 L
```

```
▸ Frame 7: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 57338, Dst Port: 80, Seq: 84, Ack: 353, Len: 0
    Source Port: 57338
    Destination Port: 80
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 84    (relative sequence number)
    Sequence number (raw): 2637494999
    [Next sequence number: 84    (relative sequence number)]
    Acknowledgment number: 353    (relative ack number)
    Acknowledgment number (raw): 335975417
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x010 (ACK)
```

# TCP Close connection



8

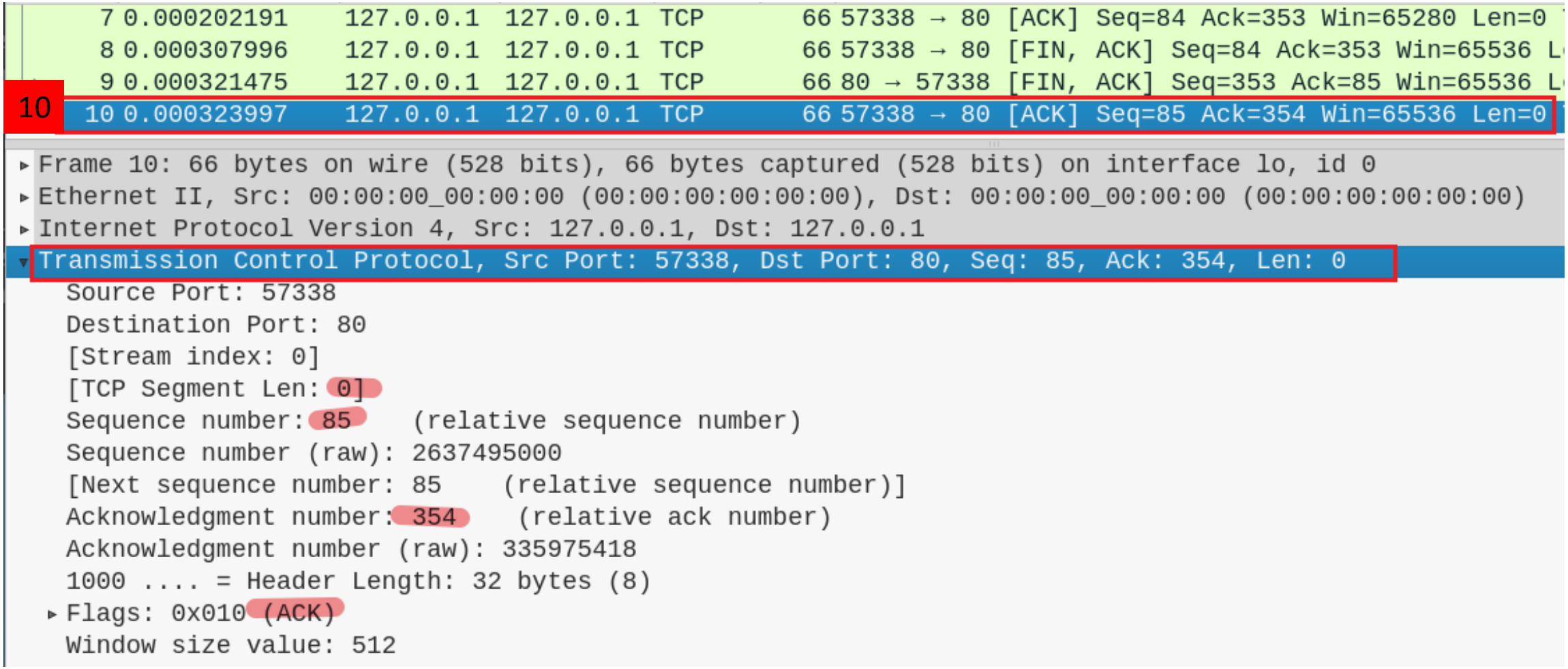
```
    7 0.000202191    127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=84 Ack=353 Win=65280 Len=0
    8 0.000307996    127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [FIN, ACK] Seq=84 Ack=353 Win=65536 L
    9 0.000321475    127.0.0.1  127.0.0.1  TCP      66 80 → 57338 [FIN, ACK] Seq=353 Ack=85 Win=65536 L
   10 0.000323997    127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=85 Ack=354 Win=65536 Len=0
```

```
▸ Frame 8: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 57338, Dst Port: 80, Seq: 84, Ack: 353, Len: 0
    Source Port: 57338
    Destination Port: 80
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 84    (relative sequence number)
    Sequence number (raw): 2637494999
    [Next sequence number: 85    (relative sequence number)]
    Acknowledgment number: 353    (relative ack number)
    Acknowledgment number (raw): 335975417
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x011 (FIN, ACK)
```

9

```
    8 0.000307996    127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [FIN, ACK] Seq=84 Ack=353 W
    9 0.000321475    127.0.0.1  127.0.0.1  TCP      66 80 → 57338 [FIN, ACK] Seq=353 Ack=85 W
   10 0.000323997    127.0.0.1  127.0.0.1  TCP      66 57338 → 80 [ACK] Seq=85 Ack=354 Win=65
```

```
▸ Frame 9: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface lo, id 0
▸ Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:0
▸ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
▾ Transmission Control Protocol, Src Port: 80, Dst Port: 57338, Seq: 353, Ack: 85, Len: 0
    Source Port: 80
    Destination Port: 57338
    [Stream index: 0]
    [TCP Segment Len: 0]
    Sequence number: 353    (relative sequence number)
    Sequence number (raw): 335975417
    [Next sequence number: 354    (relative sequence number)]
    Acknowledgment number: 85    (relative ack number)
    Acknowledgment number (raw): 2637495000
    1000 .... = Header Length: 32 bytes (8)
  ▸ Flags: 0x011 (FIN, ACK)
```



# Summary

- Think of a header as an envelope
  - Only the smallest envelope (TCP header) contains the data (HTTP data) we want to read
- An envelope describes what is inside the envelope
  - Information shown on an envelope is the metadata of the payload
- Sequence # is the byte number of the first byte of HTTP data in the TCP segment sent
  - beginning at random # or 0 (relative seq#)
- Acknowledge # is the sequence number of the next byte the receiver expects to receive
  - Seq # + size of packet + 1
- Handshaking TCPs don't have HTTP data (size=0)

# Assignment

1. Create a simple website with your name on it
2. Capture the traffic
3. Capture screens with the following information
   - the website with your name
   - ports (sender and receiver)
   - initial sequence numbers (sender and receiver)
   - the timestamps of handshaking
   - IP addresses (sender and receiver)
   - Mac addresses (sender and receiver)
   - the timestamps of HTTP request
4. Repeat tasks 2 and 3 using the following website
   - http://shinyfreshmajesticsmile.neverssl.com/online/